V Vault · User Deposit System



Պաշտպանության ուղեցույց

Ինչ տեխնոլոգիաներ ենք օգտագործել, որ ֆայլում ինչ կա, որ տողերն են կարևոր, և հավանական հարցերի պատասխանները: Ամեն պատասխան հղում է կոդի կոնկրետ ֆայլին ու տողին:

PostgreSQL · ACID | REPEATABLE READ | SELECT ... FOR UPDATE | Deadlock 40P01 | Composite index | Redis | RabbitMQ · DLQ | Streams · backpressure | Worker Threads | Cluster | JWT | Docker

wallet-service · notification-service · web

Տողերի համարները վերցված են իրական կոդից

00 Բովանդակություն

Ինչպես օգտագործել. նախ կարդա 01 բաժինը (ամբողջ պատկերը), հետո ամեն թեմայի «Ինչ ասել բանավոր» բլոկները: Հարցերը նշված են երեք մակարդակով՝ Հիմնական Միջին Խորը: Մանուշակագույն `wallet/ֆայլ.ts:42` պիտակները ցույց են տալիս, թե կոնկրետ որտեղ է դա կոդում:

01 Ինչ ենք օգտագործում
Տեխնոլոգիաների քարտեզ՝ այո / ոչ, որտեղ

02 Ճարտարապետություն
Service-ներ, deposit-ի և transfer-ի հոսքը քայլ առ քայլ

03 Transaction-ներ և ACID
BEGIN/COMMIT/ROLLBACK, isolation, lock, deadlock

04 Indexing
Composite index, leftmost prefix, EXPLAIN

05 Redis
Idempotency, rate limit, cache, report status

06 RabbitMQ
Exchange, routing key, ack, retry, DLQ

07 Streams և backpressure
CSV export cursor-ով

08 Worker Threads և Cluster
CPU-ծանր աշխատանք, scaling, self-healing

09 Auth, transfer, server
JWT, lock ordering, module loader, errors

10 Ֆայլ առ ֆայլ
Ամեն ֆայլի դերը և կարևոր տողերը

11 Խորամանկ հարցեր
Թույլ կողմեր և ինչպես ազնիվ պատասխանել

12 Demo և բառարան
Հրամաններ ցույց տալու համար, տերմիններ

ՊԱՐԶ ՕՐԻՆԱԿՈՎ

Ծառի կառուցվածքը կարճ. `wallet/` = `wallet-service/src/`, `notification/` = `notification-service/src/`: Ամեն `wallet/modules/<name>/` folder ունի նույն 5 ֆայլը՝ `router.ts` (HTTP route-եր), `service.ts` (business logic), `repository.ts` (DB access), `model.ts` (table), `middleware.ts`: Router-ը երբեք ուղիղ DB չի դիպչում, միշտ service-ի միջոցով:

01 Ինչ ենք օգտագործում

Ամենահաճախ առաջին հարցը՝ «ինչ տեխնոլոգիաներ կան նախագծում և ինչու»: Այս էջը պատասխանում է նաև «իսկ X-ը կա՞» տիպի հարցերին, օրինակ՝ BullMQ, composite index, migration:

ՏԵԽՆՈԼՈԳԻԱ	ԿԱ՞	ԻՆՉԻ ՀԱՄԱՐ	ՈՐՏԵՂ
Node.js + TypeScript	Այո	Ամբողջ backend-ը	`wallet/index.ts`
Express	Այո	HTTP server, middleware chain	`wallet/server.ts:18`
PostgreSQL 16	Այո	`accounts` և `transactions` table-եր	`docker-compose.yml:3`
Sequelize (ORM)	Այո	Model-եր, transaction-ներ, lock, isolation level	`wallet/database.ts:13`
pg + pg-query-stream	Այո	Raw cursor CSV export-ի համար (Sequelize-ը stream չի անում)	`wallet/modules/transactions/router.ts:30`
Redis	Այո	Idempotency, rate limit, page-1 cache, report status, event log	`wallet/services/redis.ts`
RabbitMQ	Այո	`deposit.created` event → notification-service, retry, DLQ	`wallet/services/rabbitmq.ts:13`
BullMQ / Bull	Ոչ	Չենք օգտագործում: Report-ը Worker Thread է, event-ները՝ RabbitMQ	—
Worker Threads	Այո	PDF report-ի CPU-ծանր հաշվարկ	`wallet/modules/reports/service.ts:26`
Cluster module	Այո	N worker process մեկ port-ի վրա + self-healing	`wallet/index.ts:45`
Node.js Streams	Այո	CSV export, backpressure	`wallet/modules/transactions/router.ts:74`
DB transaction (ACID)	Այո	Deposit, transfer	`wallet/modules/deposits/service.ts:18`
Isolation level	Այո	`REPEATABLE READ`, explicit	`wallet/modules/deposits/service.ts:21`
Row lock (FOR UPDATE)	Այո	Lost update-ի կանխում	`wallet/modules/deposits/service.ts:27`
Deadlock handling	Այո	Retry `40P01`-ի դեպքում + lock ordering transfer-ում	`wallet/utils/withDeadlockRetry.ts:8`
Composite index	Այո	`idx_user_created (user_id, created_at)`	`wallet/modules/transactions/model.ts:58`
CHECK constraint	Այո	`balance ⩾ 0`	`wallet/database.ts:69`

ՏԵԽՆՈԼՈԳԻԱ	ԿԱ՞	ԻՆՉԻ ՀԱՄԱՐ	ՈՐՏԵՂ
JWT auth	Այո	Transfer և `/accounts/me`	`wallet/utils/jwt.ts:11`
Docker / compose	Այո	3 infra + 2 service container	`docker-compose.yml`
Swagger / OpenAPI	Այո	`/docs`	`wallet/server.ts:39`
React + Vite (web)	Այո	UI, Test lab, Live logs	`web/src/App.tsx`
DB migrations	Ոչ	`sync(alter)` է, ոչ migration (տես 11 բաժինը)	`wallet/database.ts:33`
Unit / integration tests	Ոչ	Ավտոմատ test-եր չկան. ստուգումը Test lab-ով և demo script-ով է	`wallet/scripts/deadlock-demo.ts`
Outbox pattern	Ոչ	Event-ը publish է արվում commit-ից հետո, fire-and-forget	`wallet/modules/deposits/service.ts:67`

Transaction-ի մեջ ինչ կա ԱՄՓՈՓ

ACID — Այո

Deposit-ը մեկ transaction-ում փոխում է 2 table՝ `accounts.balance` UPDATE և `transactions` INSERT: Կամ երկուսն են լինում, կամ ոչ մեկը:

`wallet/modules/deposits/service.ts:35`
`wallet/modules/deposits/service.ts:43`

BEGIN / COMMIT / ROLLBACK — Այո

`sequelize.transaction(opts, async t ⇒ …)` managed ձևն է. callback-ի սկզբում BEGIN, հաջող return → COMMIT, throw → ROLLBACK:

`wallet/modules/deposits/service.ts:18`

Isolation level — Այո

`REPEATABLE READ`. ամբողջ transaction-ը տեսնում է նույն snapshot-ը: Postgres-ի default-ը `READ COMMITTED` է:

`wallet/modules/deposits/service.ts:21`
`wallet/modules/transfers/service.ts:26`

Row-level lock — Այո

`SELECT … FOR UPDATE`. նույն account-ի 2 զուգահեռ deposit-ը հերթով է անցնում, lost update չկա:

`wallet/modules/deposits/service.ts:27`

Deadlock — Այո

Postgres-ը ինքն է գտնում (error `40P01`), մեկին abort անում: Մենք retry ենք անում 3 անգամ backoff-ով: Transfer-ում lock-երը վերցվում են ըստ userId աճման կարգի, որ deadlock ընդհանրապես չլինի:

`wallet/utils/withDeadlockRetry.ts:8`
`wallet/modules/transfers/service.ts:19`
`wallet/scripts/deadlock-demo.ts:48`

Consistency — Այո

`CHECK (balance ⩾ 0)` DB մակարդակում, `DECIMAL(18,2)` փողի համար (ոչ float), FK `transactions.user_id → accounts.user_id`:

`wallet/database.ts:69`
`wallet/modules/accounts/model.ts:32`
`wallet/core/associations.ts:10`

Indexing-ի մեջ ինչ կա ԱՄՓՈՓ

INDEX	ՏԵՍԱԿ	ԻՆՉ Է ԱՆՈՒՄ	ՈՐՏԵՂԻՑ
`idx_user_created`	**Composite** B-tree `(user_id, created_at)` Այո	History query-ն `WHERE user_id = $1 ORDER BY created_at DESC` անցնում է index-ով, առանց sort-ի	`wallet/modules/transactions/model.ts:58` `wallet/database.ts:70`
`accounts_pkey`, `transactions_pkey`	Primary key (unique)	Ավտոմատ, `id` սյունակի վրա	`wallet/modules/accounts/model.ts:22`
`accounts_user_id_key`	Unique	`unique: true`-ից. մեկ user-ին մեկ account: Սրա վրա է աշխատում `FOR UPDATE`-ի `WHERE user_id`-ը	`wallet/modules/accounts/model.ts:27`
FK `transactions.user_id`	Foreign key (index չէ)	Postgres-ը FK-ի համար ավտոմատ index չի ստեղծում, բայց composite index-ի առաջին սյունակը `user_id` է, այնպես որ FK lookup-ներն էլ արագ են	`wallet/core/associations.ts:10`

ԻՆՉ ԱՍԵԼ ԲԱՆԱՎՈՐ

«Այո, composite index կա՝ `idx_user_created (user_id, created_at)`: Այն սահմանված է model-ում և, ապահովության համար, նաև raw SQL-ով `postSyncFixups()`-ում: Query-ն `WHERE user_id = ? ORDER BY created_at DESC` է, որը համապատասխանում է leftmost prefix-ին. Postgres-ն անում է `Index Scan Backward`, առանց առանձին sort-ի»:

BullMQ-ի մասին, եթե հարցնեն

BullMQ-ն չենք օգտագործում: BullMQ-ն Redis-ի վրա աշխատող *job queue* է. job-երը պահվում են Redis-ում, ունի retry, delay, priority, և job-ը չի կորչում process-ի crash-ի դեպքում: Մեզ մոտ նույն խնդիրները լուծված են ուրիշ գործիքներով, քանի որ առաջադրանքը հենց դրանք էր պահանջում.

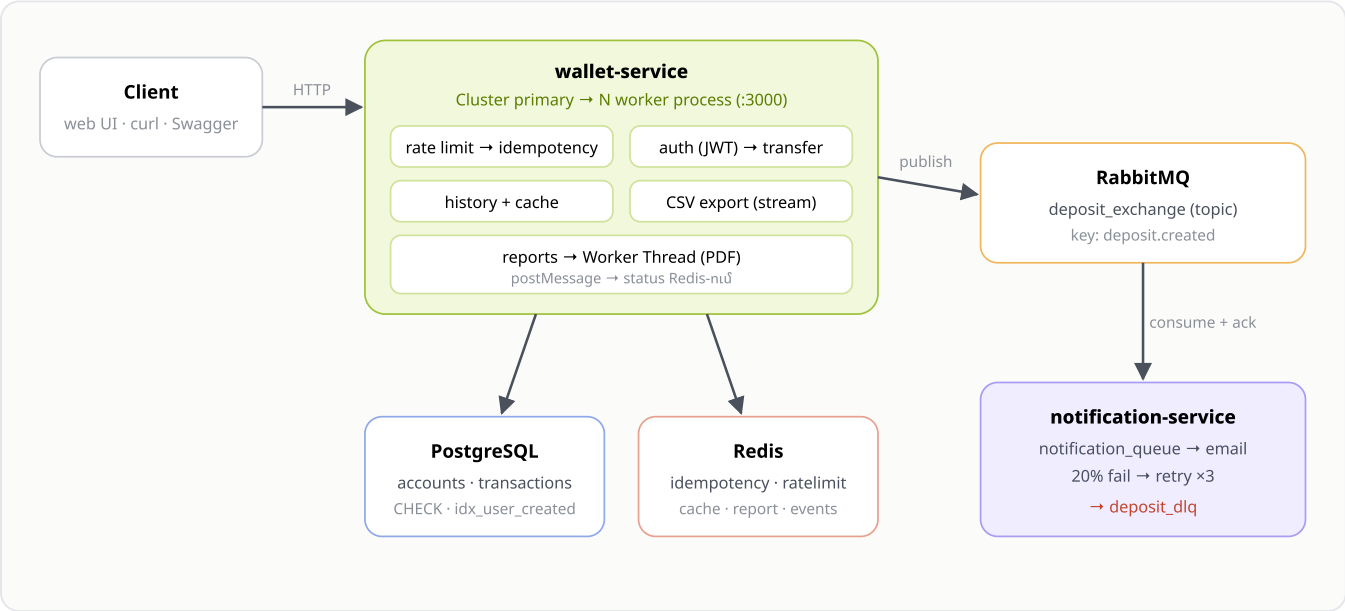
- **Ծանր report** → `worker_threads` (առաջադրանքը պահանջում էր Worker Thread) `wallet/modules/reports/service.ts:26`
- **Service-ների միջև event-ներ, retry, DLQ** → RabbitMQ `notification/index.ts:12`

ՈՒՇԱԴՐՈՒԹՅՈՒՆ

Ազնիվ տարբերությունը. եթե wallet-service-ի process-ը crash լինի report-ի ընթացքում, Worker Thread-ն էլ է մեռնում, և Redis-ում status-ը մնում է `processing`: BullMQ-ով job-ը կմնար queue-ում, և այլ worker կվերցներ այն: Եթե հարցնեն «ինչպես կլավացնեիր», կարող ես ասել. «Production-ում report-ը կդնեի BullMQ queue-ի մեջ, իսկ BullMQ worker-ը ներսում կօգտագործեր worker thread»:

02 Ճարտարապետություն

Երկու անկախ service, որոնք իրար կոդ չեն import անում: Միակ «պայմանագիրը» RabbitMQ-ի `deposit_exchange`-ը և message-ի ձևն է:



POST /deposit, քայլ առ քայլ

1. **Rate limit.** Redis `INCR ratelimit:deposit:<userId>:<րոպե>`. 10-ից ավելի → `429`: `wallet/core/rateLimiter.middleware.ts:15`
2. **Idempotency.** `Idempotency-Key` header-ը պարտադիր է: Redis-ում `SET NX`-ով «զբաղեցնում» ենք key-ը. եթե key-ն արդեն կա, վերադարձնում ենք պահված պատասխանը կամ `409`: `wallet/core/idempotency.middleware.ts:39`
3. **Validation.** `userId`, `amount > 0`: `wallet/modules/deposits/router.ts:17`
4. **BEGIN** `REPEATABLE READ`: `wallet/modules/deposits/service.ts:21`
5. **SELECT ... FOR UPDATE** account row-ի վրա: `wallet/modules/deposits/service.ts:27`
6. **UPDATE** balance + **INSERT** transaction row: `wallet/modules/deposits/service.ts:35`
7. **COMMIT**. callback-ը հաջողությամբ վերադարձավ: Error-ի դեպքում ROLLBACK, իսկ deadlock-ի դեպքում retry:
8. **Publish** `deposit.created` RabbitMQ (fire-and-forget): `wallet/modules/deposits/service.ts:67`

POST /transfer, քայլ առ քայլ

1. **authGuard.** JWT-ից վերցնում ենք `authUserId`: Sender-ը token-ից է, ոչ body-ից: `wallet/core/auth.middleware.ts:27`
2. **Self-transfer** → `400`: `wallet/modules/transfers/service.ts:16`
3. **Lock ordering.** Երկու account-ը lock ենք անում ըստ userId աճման կարգի, ոչ թե «from → to»: `wallet/modules/transfers/service.ts:19`
4. **Ստուգում**. account-ները կան, balance-ը բավարար է, թե ոչ → `404/400`: `wallet/modules/transfers/service.ts:42`
5. **Debit** sender, **credit** receiver: `wallet/modules/transfers/service.ts:49`
6. **2 INSERT**՝ `transfer_out` և `transfer_in`: `wallet/modules/transfers/service.ts:71`
7. **COMMIT**. 2 UPDATE + 2 INSERT ատոմիկ: Ցանկացած քայլում error → ամեն ինչ ROLLBACK:

ՊԱՐԶ ՕՐԻՆԱԿՈՎ

Transfer-ը փողը չի «ստեղծում» կամ «կորցնում»՝ user-ների balance-ների գումարը մնում է նույնը: Test lab-ի «Total money across users is conserved» ստուգումը հենց դա է ստուգում:

9 **Save** 201 պատասխանը Redis-ում 24 ժամով (idempotent replay): `wallet/modules/deposits/router.ts:3`

Endpoint-ների ցուցակ

METHOD · PATH	ԻՆՉ Է ԱՆՈՒՄ	ՊԱՇՏՊԱՆՈՒԹՅՈՒՆ	ԿՈԴ
`POST /auth/login`	Demo login (`userN/userN`) → JWT	—	`wallet/modules/auth/service.ts:8`
`GET /accounts/me`	Ընթացիկ balance	JWT	`wallet/modules/accounts/router.ts:11`
`POST /deposit`	Գումար ավելացնել	Rate limit + Idempotency-Key	`wallet/modules/deposits/router.ts:12`
`POST /transfer`	Փոխանցում այլ user-ի	JWT	`wallet/modules/transfers/router.ts:12`
`GET /transactions/:userId`	History, paginated, page 1-ը cache-ով	—	`wallet/modules/transactions/router.ts:13`
`GET /transactions/:userId/export`	CSV stream	—	`wallet/modules/transactions/router.ts:24`
`POST /reports/generate`	202 + reportId, Worker Thread	—	`wallet/modules/reports/router.ts:11`
`GET /reports/:id/status`, `/download`	Status Redis-ից, PDF download	—	`wallet/modules/reports/router.ts:26`
`GET/DELETE /events`	Trace log UI-ի համար (Test lab, Logs)	—	`wallet/modules/events/router.ts:6`
`GET /health`, `/docs`	Healthcheck, Swagger UI	—	`wallet/server.ts:35`

03 Transaction-ներ և ACID

Ամենակարևոր թեման. այստեղ ամենաշատ «ինչո՞ւ» հարցերը կլինեն:

wallet/modules/deposits/service.ts · deposit-ի transaction-ը — տողեր 18-61

```
18      const result = await withDeadlockRetry(() => sequelize.transaction(
19
20        {
21          isolationLevel: SequelizeTransaction.ISOLATION_LEVELS.REPEATABLE_READ
22        },
23        async (t) => {
24          const account = await Account.findOne({
25            where:       { userId },
26            transaction: t,
27            lock:        t.LOCK.UPDATE, // SELECT ... FOR UPDATE
28          });
29          if (!account) throw new NotFound(`No account found for userId ${userId}`);
30          trace('info', 'deposit.lock', `Locked account user${userId} (FOR UPDATE)`, { users });
31
32          const before = account.balance;
33          const newBalance = (Number(account.balance) + amount).toFixed(2);
34
35          await account.update({
36            balance: newBalance
37          }, {
38            transaction: t
39          });
40          trace('info', 'deposit.credit', `Credit user${userId}: ${before} → ${newBalance}`, { users, data:
    { balance: newBalance } });
41          faultPoint('deposit.after-credit');
42
43          const transaction = await TransactionModel.create({
44            userId,
45            amount:       amount.toFixed(2),
46            type:         'deposit',
47            balanceAfter: newBalance,
48          }, {
49            transaction: t
50          });
51
52          trace('info', 'deposit.row', `Inserted deposit #${transaction.id} (user${userId})`, { users, data:
    { id: transaction.id } });
53          faultPoint('deposit.before-commit');
54
55          return {
56            transaction, balance: newBalance
57          };
58        }
59      )).catch((err) => {
60        trace('error', 'deposit.rollback', `ROLLBACK deposit ${label}: ${(err as Error).message}`, { users
    });
61        throw err;
```

Կանաչ տողերը. isolation level, row lock, 1-ին գրառում (UPDATE), 2-րդ գրառում (INSERT): `trace(...)` և `faultPoint(...)` տողերը UI-ի Logs/Test lab-ի համար են և business logic-ի վրա չեն ազդում:

ԻՆՉ ԱՍԵԼ ԲԱՆԱՎՈՐ

«Deposit-ը փոխում է երկու table՝ balance-ը և history-ն: Երկուսն էլ նույն `sequelize.transaction`-ի t-ի մեջ են, այսինքն մեկ BEGIN...COMMIT-ի: Եթե INSERT-ը fail լինի, Sequelize-ը ROLLBACK է անում, և UPDATE-ն էլ հետ է գնում: Balance-ը առանց history row-ի չի կարող մնալ»:

ՀԻՄՆԱԿԱՆ **Ի՞նչ է ACID-ը, և որտեղ է այն ձեր նախագծում:**

Atomicity՝ կամ բոլոր փոփոխությունները, կամ ոչ մեկը: **C**onsistency՝ DB-ն միշտ մնում է կանոնների մեջ (`CHECK balance ⩾ 0`, FK, NOT NULL): **I**solation՝ զուգահեռ transaction-ները իրար «կիսատ» վիճակը չեն տեսնում: **D**urability՝ COMMIT-ից հետո տվյալը չի կորչում (Postgres-ի WAL):

Մեզ մոտ ACID-ը deposit-ում և transfer-ում է՝ մի քանի գրառում մեկ transaction-ում:

ՈՐՏԵՂ `wallet/modules/deposits/service.ts:18` `wallet/modules/transfers/service.ts:24` `wallet/database.ts:69`

ՄԻՋԻՆ **Որտեղ են BEGIN, COMMIT, ROLLBACK բառերը կոդում: Ես չեմ տեսնում:**

Մենք օգտագործում ենք Sequelize-ի **managed transaction**-ը՝ `sequelize.transaction(options, async (t) ⇒ { … })`: Sequelize-ը callback-ից առաջ ուղարկում է `START TRANSACTION` (BEGIN), callback-ի հաջող ավարտին՝ `COMMIT`, իսկ եթե callback-ը throw է անում՝ `ROLLBACK`, և error-ը շարունակում է վեր գնալ:

Կարևորն այն է, որ ամեն query ստանա `{ transaction: t }`: Առանց դրա query-ն կգնար ուրիշ connection-ով, transaction-ից դուրս:

ՈՐՏԵՂ `wallet/modules/deposits/service.ts:18` `wallet/modules/deposits/service.ts:38`

ՄԻՋԻՆ **Ինչպե՞ս ապացուցեցիք, որ rollback-ն իսկապես աշխատում է:**

Ավելացրել ենք **fault injection**: Եթե request-ն ունի `X-Fault: transfer.after-debit` header (և `FAULT_INJECTION=true`), `faultPoint()`-ը throw է անում transaction-ի մեջտեղում: Test lab-ի սցենարը ստուգում է, որ դրանից հետո balance-ները չեն փոխվել: Server trace-ում երևում է `Debit …`, հետո `ROLLBACK`:

Կա նաև `transfer.before-commit` կետը. այնտեղ երկու INSERT-ն էլ արդեն արված է, ID-ներն էլ ստացված են, բայց rollback-ից հետո row-երը չեն երևում: Մի հետաքրքիր դետալ. sequence-ի ID-ները «այրվում» են, և history-ում ID-ների մեջ բացեր են առաջանում: Դա նորմալ է, sequence-ը transaction-ով չի rollback լինում:

ՈՐՏԵՂ `wallet/utils/trace.ts:88` `wallet/modules/transfers/service.ts:57` `web/src/lab/scenarios.ts:104`

ՀԻՄՆԱԿԱՆ **Ինչո՞ւ է balance-ը DECIMAL, ոչ FLOAT:**

Float-ը binary է, և `0.1 + 0.2 = 0.30000000000000004`: Փողի համար սա անթույլատրելի է: `DECIMAL(18,2)`-ը ճշգրիտ տասնորդական է: Sequelize-ը DECIMAL-ը վերադարձնում է string, դրա համար model-ում `balance: string` է:

Թույլ կետ. service-ում հաշվարկը `Number(balance) + amount` է, այսինքն JS float, որից հետո `toFixed(2)`: Ավելի ճիշտ կլիներ `UPDATE … SET balance = balance + $1` կամ decimal գրադարան (տես 11 բաժինը):

ՈՐՏԵՂ `wallet/modules/accounts/model.ts:32` `wallet/modules/deposits/service.ts:33`

ՀԻՄՆԱԿԱՆ **Ինչի՞ համար է CHECK (balance >= 0), եթե deposit-ը միշտ ավելացնում է:**

Դա DB-ի մակարդակի վերջին պաշտպանությունն է: Եթե կոդում bug լինի (transfer, ապագա withdrawal), Postgres-ը կմերժի UPDATE-ը, և ամբողջ transaction-ը կrollback-վի: Transfer-ում նախ ստուգում ենք կոդում (`Insufficient balance`), իսկ CHECK-ը ապահովագրում է:

Այն ավելացվում է raw SQL-ով, քանի որ Sequelize-ի `sync()`-ը CHECK constraint ինքնուրույն չի ստեղծում:

ՈՐՏԵՂ `wallet/database.ts:69` `wallet/modules/transfers/service.ts:42`

Isolation level REPEATABLE READ

LEVEL	DIRTY READ	NON-REPEATABLE READ	PHANTOM	ՄԵԿՆԱԲԱՆՈՒԹՅՈՒՆ
READ UNCOMMITTED	Postgres-ում ոչ	Այո	Այո	Postgres-ում իրականում նույնն է, ինչ READ COMMITTED
READ COMMITTED (default)	Ոչ	Այո	Այո	Ամեն statement տեսնում է թարմ snapshot
REPEATABLE READ (մերը)	Ոչ	Ոչ	Postgres-ում ոչ	Ամբողջ transaction-ը տեսնում է մեկ snapshot. կոնֆլիկտի դեպքում `40001`
SERIALIZABLE	Ոչ	Ոչ	Ոչ	Ամենախիստը, ավելի շատ `40001` և retry

ՀԻՄՆԱԿԱՆ **Ինչո՞ւ եք ընտրել REPEATABLE READ:**

Lost update-ից իրականում պաշտպանում է `FOR UPDATE`-ը, և դա կաշխատեր նաև READ COMMITTED-ով: REPEATABLE READ-ը ավելացրել ենք որպես «ապահովագրություն». transaction-ի բոլոր read-երը տեսնում են նույն snapshot-ը: Եթե ապագայում ավելացնենք լրացուցիչ ստուգումներ (daily limit, fraud check), դրանք transaction-ի կեսին չեն տեսնի ուրիշ տվյալ: Բացի այդ, առաջադրանքի bonus-ը հենց explicit isolation level-ն էր պահանջում:

ՈՐՏԵՂ `wallet/modules/deposits/service.ts:21` `ARCHITECTURE.md:5`

ԽՈՐԸ **Ի՞նչ է լինում REPEATABLE READ + FOR UPDATE-ի դեպքում, երբ մեկ ուրիշը արդեն փոխել է row-ը:**

Երկրորդ transaction-ը սպասում է lock-ին: Երբ առաջինը COMMIT է անում, Postgres-ը տեսնում է, որ row-ը փոխվել է այս transaction-ի snapshot-ից հետո, և REPEATABLE READ-ում նետում է `40001 could not serialize access due to concurrent update`: READ COMMITTED-ում նույն դեպքում պարզապես կկարդար նոր տարբերակը և կշարունակեր:

Մեր `withDeadlockRetry`-ը retry է անում միայն `40P01`-ի դեպքում, ոչ `40001`-ի: Այդ պատճառով շատ զուգահեռ transfer-ների ժամանակ մի մասը վերադարձնում է `500`: Դա երևաց Test lab-ի concurrency սցենարներում: Լուծումը կամ READ COMMITTED-ն է (lock-ն արդեն պաշտպանում է), կամ `40001`-ն էլ retry անելը:

ՈՐՏԵՂ `wallet/utils/withDeadlockRetry.ts:21` `web/src/lab/scenarios.ts:176`

Locking և race condition

Ինչու է պետք SELECT … FOR UPDATE

```
-- առանց lock-ի (lost update)
T1: SELECT balance → 100          T2: SELECT balance → 100
T1: UPDATE balance = 150          T2: UPDATE balance = 150   ← T1-ի +50-ը կորավ
-- FOR UPDATE-ով
T1: SELECT … FOR UPDATE → 100     T2: SELECT … FOR UPDATE → սպասում է…
T1: UPDATE 150, COMMIT            T2: (lock ստացավ) → 150, UPDATE 200 ✓
```

ՀԻՄՆԱԿԱՆ **Ի՞նչ է lost update-ը, և ինչպե՞ս եք կանխում:**

Երկու request միաժամանակ կարդում են նույն balance-ը, ավելացնում են իրենց գումարը և գրում: Արդյունքում մեկի գրածը ջնջում է մյուսինը: Մենք account row-ը կարդում ենք `lock: t.LOCK.UPDATE`-ով, ինչը SQL-ում `SELECT … FOR UPDATE` է: Երկրորդ transaction-ը սպասում է, մինչև առաջինը COMMIT անի, և հետո տեսնում է արդեն թարմացված balance-ը:

Lock-ը **row-level** է, այսինքն ուրիշ user-ների deposit-ները չեն սպասում:

ՈՐՏԵՂ `wallet/modules/deposits/service.ts:27` `PROJECT_GUIDE.md:115`

ՄԻՋԻՆ **Ինչո՞ւ FOR UPDATE, ոչ թե պարզապես UPDATE balance = balance + x:**

Atomic UPDATE-ն էլ կլուծեր lost update-ը մեկ statement-ով, և դա նույնիսկ ավելի պարզ է: Մեզ պետք էր նաև **balance_after**-ը գրել history-ում և transfer-ում ստուգել «բավարա՞ր է balance-ը» նախքան փոխելը: Դրա համար նախ կարդում ենք lock-ով, հետո որոշում ենք: Բացի այդ, առաջադրանքն ուղիղ պահանջում էր `SELECT … FOR UPDATE`:

ՈՐՏԵՂ `wallet/modules/transfers/service.ts:42`

Deadlock

wallet/utils/withDeadlockRetry.ts — տողեր 13–27

```ts
export async function withDeadlockRetry<T>(fn: () => Promise<T>, attempts = 3): Promise<T> {
  for (let attempt = 1; attempt <= attempts; attempt += 1) {
    try {
      return await fn();
    } catch (err) {
      const code = (err as { original?: { code?: string }; parent?: { code?: string } })?.original?.code
        ?? (err as { parent?: { code?: string } })?.parent?.code;

      if (code === '40P01' && attempt < attempts) {
        logger.warn(`Deadlock detected, retrying (attempt ${attempt}/${attempts})`);
        trace('warn', 'deadlock.retry', `Postgres aborted a deadlocked transaction (40P01) — retrying ${attempt}/${attempts}`);
        await sleep(50 * attempt);
        continue;
      }
      throw err;
```

ՀԻՄՆԱԿԱՆ **Ի՞նչ է deadlock-ը, և կարո՞ղ է deposit-ը deadlock լինել:**

Deadlock-ը այն է, երբ T1-ը պահում է A lock-ը և սպասում B-ին, իսկ T2-ը պահում է B-ն և սպասում A-ին: Ոչ մեկը չի կարող շարունակել: Postgres-ը դա հայտնաբերում է (`deadlock_timeout`, default 1s), abort է անում մեկին `40P01` error-ով, իսկ մյուսը շարունակում է:

Deposit-ը deadlock չի կարող լինել, քանի որ lock է անում միայն **մեկ** row: Երկու deposit-ը կարող է միայն սպասել իրար: Deadlock-ի համար պետք է ամեն transaction-ում առնվազն 2 lock՝ հակառակ հերթականությամբ: Դա հենց transfer-ի դեպքն է, և այդ պատճառով այնտեղ կա lock ordering:

ՈՐՏԵՂ `wallet/scripts/deadlock-demo.ts:2` `wallet/utils/withDeadlockRetry.ts:8`

ՄԻՋԻՆ **Ինչպե՞ս եք կանխում deadlock-ը transfer-ում:**

Lock ordering. երկու account-ը միշտ lock ենք անում փոքր userId-ից դեպի մեծը, անկախ նրանից, թե ով է ուղարկողը: user2→user3 և user3→user2 transfer-ները երկուսն էլ նախ lock են անում user2-ը, հետո user3-ը, ուստի հակառակ հերթականություն չի լինում: `withDeadlockRetry`-ը մնում է որպես backstop:

ՈՐՏԵՂ `wallet/modules/transfers/service.ts:19` `wallet/modules/transfers/service.ts:30`

ՀԻՄՆԱԿԱՆ **Ինչպե՞ս կարող եմ ցույց տալ deadlock-ը:**

`cd wallet-service && npm run demo:deadlock`: Script-ը միաժամանակ բացում է T1 (lock 1, 300ms սպասում, lock 2) և T2 (lock 2, 300ms սպասում, lock 1): Postgres-ը abort է անում մեկին, log-ում երևում է `Deadlock detected, retrying`, retry-ից հետո երկուսն էլ հաջող են ավարտվում:

ՈՐՏԵՂ `wallet/scripts/deadlock-demo.ts:48` `wallet/scripts/deadlock-demo.ts:25`

ՄԻՋԻՆ **Ինչո՞ւ է retry-ը backoff-ով (50ms × attempt), և ինչո՞ւ միայն 3 անգամ:**

Եթե անմիջապես retry անենք, երկու transaction-ը կարող են նորից նույն պահին բախվել: Փոքր սպասումը (50, 100ms) նրանց «ցրում» է ժամանակի մեջ: 3 փորձից հետո error-ը վերադարձնում ենք client-ին, որպեսզի անվերջ loop չլինի:

ՈՐՏԵՂ `wallet/utils/withDeadlockRetry.ts:24` `wallet/utils/withDeadlockRetry.ts:13`

04 Indexing

Հարց, որ շատ հաճախ են տալիս. «ինչ index կա, composite կա՞, ինչու է այդ հերթականությամբ»:

wallet/modules/transactions/model.ts — տողեր 50-59

```
50    tableName:   'transactions',
51    underscored: true,
52    updatedAt:   false,
53    // The leftmost prefix of this index is user_id, so
54    // `WHERE user_id = $1 ORDER BY created_at DESC` is satisfied entirely
55    // by an index scan — no separate sort step, no full table scan.
56    indexes: [
57      {
58        name:   'idx_user_created',
59        fields: ['user_id', 'created_at'],
```

wallet/database.ts — տողեր 66-71

```
66    private async postSyncFixups() {
67      const statements = [
68        `ALTER TABLE accounts
69           ADD CONSTRAINT balance_non_negative CHECK (balance ⩾ 0)`,
70        `CREATE INDEX IF NOT EXISTS idx_user_created
71           ON transactions (user_id, created_at)`,
```

wallet/core/repository.ts · history query-ն, որ օգտագործում է index-ը — տողեր 32-40

```
32    public getAll(filter: Record<string, unknown>) {
33      const { limit = 10, page = 1, ...where } = filter;
34      const offset = ((page as number) - 1) * (limit as number);
35
36      return this.model.findAndCountAll({
37        where: where as WhereOptions<Attributes<M>>,
38        limit: limit as number,
39        offset,
40        order: [['createdAt', 'DESC']],
```

Իրական EXPLAIN մեր DB-ից

```
-- SET enable_seqscan = off; EXPLAIN SELECT * FROM transactions
--   WHERE user_id = 1 ORDER BY created_at DESC LIMIT 20;
Limit
  →  Index Scan Backward using idx_user_created on transactions
       Index Cond: (user_id = 1)
```

ՀԻՄՆԱԿԱՆ **Ի՞նչ է composite index-ը, և ինչո՞ւ (user_id, created_at), ոչ (created_at, user_id):**

Composite index-ը մի քանի սյունակի վրա է, և B-tree-ն դասավորված է նախ առաջին սյունակով, հետո երկրորդով: Կարելի է պատկերացնել հեռախոսագրքի պես. նախ ազգանուն, հետո անուն:

Leftmost prefix rule. index-ը կարող է օգտագործվել, երբ query-ն ֆիլտրում է **ձախից** սկսվող սյունակներով: Մեր query-ն `WHERE user_id = ?` է (առաջին սյունակը՝ հավասարությամբ) + `ORDER BY created_at` (երկրորդը): Postgres-ն անմիջապես գտնում է այդ user-ի հատվածը, որն արդեն դասավորված է ըստ ժամանակի, և պետք է միայն վերցնել 20 հատ:

Եթե լիներ `(created_at, user_id)`, user-ի row-երը ցրված կլինեին ամբողջ index-ով մեկ, և Postgres-ը ստիպված կլիներ անցնել շատ ավելի մեծ մաս:

ՈՐՏԵՂ `wallet/modules/transactions/model.ts:58` `wallet/core/repository.ts:40`

ՄԻՋԻՆ **Index-ը ASC է, իսկ query-ն DESC: Դա խնդիր չէ՞:**

Ոչ: B-tree-ն կարելի է կարդալ երկու ուղղությամբ էլ: EXPLAIN-ում դա երևում է որպես `Index Scan Backward`: Առանձին `DESC` index պետք է միայն խառը ուղղությունների դեպքում, օրինակ `ORDER BY a ASC, b DESC`:

ՈՐՏԵՂ `wallet/database.ts:71`

ԽՈՐԸ **EXPLAIN-ում ես Seq Scan եմ տեսնում, ոչ Index Scan: Index-ը չի՞ աշխատում:**

Աշխատում է: Demo DB-ում ընդամենը մոտ 100 row կա, և planner-ը հաշվում է, որ ամբողջ փոքր table-ը կարդալն ավելի էժան է, քան index-ով քայլելը: Դա ճիշտ որոշում է: Ապացուցելու համար կարելի է մուտքագրել `SET enable_seqscan = off;`, և կերևա `Index Scan Backward using idx_user_created`: Կարելի է նաև ավելացնել տասնյակ հազարավոր row, և planner-ը ինքն կընտրի index-ը:

ՄԻՋԻՆ **Ինչո՞ւ է index-ը սահմանված երկու տեղում՝ model-ում և raw SQL-ում:**

Model-ի `indexes`-ը Sequelize-ին ասում է ստեղծել index-ը `sync()`-ի ժամանակ: `postSyncFixups()`-ի `CREATE INDEX IF NOT EXISTS`-ը ապահովագրություն է, որ index-ն անպայման լինի, նույնիսկ եթե sync-ը բաց թողնի: `IF NOT EXISTS`-ի շնորհիվ երկրորդ անգամ ոչինչ չի անում: Ազնիվ ասած, մեկը բավական էր:

ՈՐՏԵՂ `wallet/modules/transactions/model.ts:56` `wallet/database.ts:70`

ԽՈՐԸ **Ի՞նչ index-ներ կան accounts table-ում:**

`accounts_pkey` (id) և **unique** index `user_id`-ի վրա (`unique: true`): Unique index-ը և՛ արգելում է մեկ user-ին երկու account ունենալ, և՛ արագացնում է `WHERE user_id = ?`-ը, որ օգտագործվում է `FOR UPDATE`-ում:

> ՈՒՇԱԴՐՈՒԹՅՈՒՆ
> Իրական DB-ում user_id-ի վրա 32 կրկնվող unique index կա (`accounts_user_id_key`, `…key1` … `…key31`): Սա Sequelize-ի հայտնի bug-ն է. `sync({ alter: true })` + `unique: true` ամեն restart-ի ժամանակ նոր constraint է ավելացնում: Տես 11 բաժինը, թե ինչպես պատասխանել:

ՈՐՏԵՂ `wallet/modules/accounts/model.ts:27` `wallet/database.ts:33`

ՄԻՋԻՆ **Pagination-ը ինչպե՞ս է աշխատում, և ի՞նչ թույլ կողմ ունի:**

Offset pagination. `LIMIT 20 OFFSET (page-1)*20` + `COUNT(*)` total-ի համար (`findAndCountAll`): Թույլ կողմ. մեծ OFFSET-ի դեպքում Postgres-ը միևնույնն է անցնում բաց թողնված row-երով, իսկ COUNT-ը ամեն անգամ հաշվում է: Ավելի լավ տարբերակը **keyset (cursor) pagination**-ն է՝ `WHERE user_id = ? AND created_at < :last ORDER BY created_at DESC LIMIT 20`, որը նույն index-ով միշտ արագ է:

ՈՐՏԵՂ `wallet/core/repository.ts:34` `wallet/core/repository.ts:36`

05 Redis

Redis-ը մեզ մոտ 5 տարբեր դեր ունի: Բոլորն էլ Redis-ում են, ոչ memory-ում, քանի որ Cluster-ի բոլոր worker process-երը պետք է նույն տվյալը տեսնեն:

KEY PATTERN	ԴԵՐ	REDIS ՀՐԱՄԱՆ	TTL
`idempotency:<key>`	Idempotency claim + պահված պատասխան	`SET NX EX`, `SET EX`, `DEL`	60s claim, 24h արդյունք
`ratelimit:deposit:<userId>:<bucket>`	Fixed-window rate limit	`INCR` + `EXPIRE`	60s
`transactions:page1:<userId>:<limit>`	Cache-aside, history-ի 1-ին էջը	`GET` / `SET EX`	30s
`report:<reportId>`	Report-ի status (processing/done/error)	`SET` / `GET`	չկա
`events:log`, `events:seq`	Trace log UI-ի համար	`INCR`, `LPUSH` + `LTRIM`, `LRANGE`	վերջին 1000-ը

Idempotency

wallet/core/idempotency.middleware.ts — տողեր 16–56

```
16  export const idempotencyGuard: MiddlewareHandler = async (req, res, next) => {
17    const key = req.headers['idempotency-key'];
18    const users = Number(req.body?.userId) ? [Number(req.body.userId)] : undefined;
19    if (!key || typeof key !== 'string') {
20      trace('warn', 'idempotency.missing', 'Rejected: Idempotency-Key header is missing', { users });
21      return next(new BadRequest('Idempotency-Key header is required'));
22    }
23
24    const cacheKey = cacheKeyFor(key);
25    const existing = await redis.get<IdempotencyRecord>(cacheKey);
26    if (existing) {
27      if (existing.processing) {
28        trace('warn', 'idempotency.in-flight', `Key ${key} is still being processed → 409`, { users, data: { key } });
29        res.status(409).json({
30          ok: false, status: 409, message: 'A request with this Idempotency-Key is still being processed'
31        });
32        return;
33      }
34      trace('info', 'idempotency.replay', `Key ${key} already completed → replaying stored ${existing.status ?? 200} response, no new write`, { users, data: { key } });
35      res.status(existing.status ?? 200).json(existing.body);
36      return;
37    }
38
39    const claimed = await redis.setNXWithTTL(
40      cacheKey,
41      JSON.stringify({
42        processing: true
43      }),
44      IDEMPOTENCY_LOCK_TTL_SECONDS
45    );
46    if (!claimed) {
```

```
    trace('warn', 'idempotency.in-flight', `Key ${key} lost the SET NX race → 409`, { users, data: { key } });
    res.status(409).json({
      ok: false, status: 409, message: 'A request with this Idempotency-Key is still being processed'
    });
    return;
  }

  trace('info', 'idempotency.claim', `Claimed key ${key} (SET NX)`, { users, data: { key } });
  req.idempotencyKey = cacheKey;
  next();
```

ԻՆՉ ԱՍԵԼ ԲԱՆԱՎՈՐ

«Client-ը ամեն deposit-ի համար ստեղծում է unique key (UUID): Եթե network-ը կտրվի և client-ը նույն key-ով կրկնի, server-ը չի անի երկրորդ deposit, այլ կվերադարձնի նույն պատասխանը նույն `transactionId`-ով: Key-ը «զբաղեցնում» ենք `SET NX`-ով, որը ատոմիկ է. երկու զուգահեռ request-ից միայն մեկը կհաղթի, իսկ մյուսը կստանա 409»:

ՄԻՋԻՆ **Ինչո՞ւ SET NX, և ոչ թե GET, հետո SET:**

GET → SET-ը երկու առանձին հրաման է, և նրանց միջև երկրորդ request-ը կարող է նույնպես GET անել, տեսնել, որ key-ը չկա, և երկուսն էլ deposit կանեն: Սա check-then-act race-ն է: `SET key value NX EX 60`-ը մեկ ատոմիկ հրաման է՝ «գրիր միայն եթե չկա»: Redis-ը single-threaded է հրամանների կատարման մեջ, ուստի միայն մեկը կստանա `OK`: Test lab-ի «5 parallel requests, one key» սցենարում արդյունքը 1×201 և 4×409 է:

ՈՐՏԵՂ `wallet/services/redis.ts:45` `wallet/core/idempotency.middleware.ts:47`

ՀԻՄՆԱԿԱՆ **Ի՞նչ է լինում, եթե deposit-ը fail լինի (DB error):**

Router-ի `catch`-ը կանչում է `releaseIdempotencyClaim`, որը ջնջում է key-ը: Նույն key-ով retry-ը կանցնի նորմալ: Եթե key-ը չջնջեինք, client-ը 60 վայրկյան 409 կստանար: Test lab-ի «Failed deposit, then retry with the same key» սցենարը հենց սա է ստուգում:

ՈՐՏԵՂ `wallet/modules/deposits/router.ts:31` `wallet/core/idempotency.middleware.ts:66`

ՄԻՋԻՆ **Ինչի՞ համար են երկու TTL-ը՝ 60s և 24h:**

60s (processing claim). եթե process-ը crash լինի deposit-ի ընթացքում, key-ը ընդմիշտ «processing» չի մնա, այլ 60 վայրկյանից հետո կջնջվի: **24h (արդյունք)**. այսքան ժամանակ retry-ը կստանա նույն պատասխանը: Սա ողջամիտ պատուհան է client-ի retry-երի համար, և Redis-ը չի լցվում ընդմիշտ:

ՈՐՏԵՂ `wallet/utils/variables.ts:30` `wallet/utils/variables.ts:31`

Rate limiting

wallet/core/rateLimiter.middleware.ts — տողեր 8-26

```ts
export const depositRateLimiter: MiddlewareHandler = async (req, res, next) => {
  const userId = req.body?.userId;
  if (!userId) return next(new BadRequest('userId is required'));

  const windowBucket = Math.floor(Date.now() / (RATE_LIMIT_WINDOW_SECONDS * 1000));
  const key = `ratelimit:deposit:${userId}:${windowBucket}`;

  const count = await redis.incrWithWindow(key, RATE_LIMIT_WINDOW_SECONDS);
  if (count > RATE_LIMIT_MAX) {
    trace('warn', 'ratelimit.blocked', `user${userId}: deposit #${count} in this ${RATE_LIMIT_WINDOW_SECONDS}s window (max ${RATE_LIMIT_MAX}) → 429`, {
      users: [Number(userId)], data: { count }
    });
    res.status(429).json({
      ok: false, status: 429, message: 'Too many deposit requests — try again in a minute'
    });
    return;
  }

  next();
```

wallet/services/redis.ts — տողեր 55-60

```ts
  public async incrWithWindow(key: string, windowSeconds: number): Promise<number> {
    const count = await this.client.incr(key);
    if (count === 1) {
      await this.client.expire(key, windowSeconds);
    }
    return count;
```

ՀԻՄՆԱԿԱՆ **Ինչպե՞ս է աշխատում rate limiter-ը:**

Fixed window: Ժամանակը բաժանում ենք 60 վայրկյանանոց «դույլերի»՝ `Math.floor(now / 60000)`: Ամեն request-ի ժամանակ `INCR` ենք անում `ratelimit:deposit:<userId>:<bucket>`-ը: Եթե արդյունքը 1 է (նոր պատուհան), `EXPIRE 60` ենք դնում: 10-ից ավելիի դեպքում վերադարձնում ենք 429: Key-ի մեջ bucket-ը կա, ուստի նոր րոպեն ինքնաբերաբար սկսվում է 0-ից:

ՈՐՏԵՂ `wallet/core/rateLimiter.middleware.ts:12` `wallet/services/redis.ts:57`

ԽՈՐԸ **INCR-ը և EXPIRE-ը երկու հրաման են: Ատոմի՞կ է:**

Ամբողջությամբ ատոմիկ չէ: Եթե process-ը crash լինի INCR-ից հետո և EXPIRE-ից առաջ, key-ը կմնա առանց TTL-ի: Բայց key-ի անունի մեջ արդեն կա րոպեի bucket-ը, ուստի հաջորդ րոպեին կօգտագործվի նոր key, և user-ը չի բլոկվի ընդմիշտ: Միայն հին key-ը կմնա Redis-ում որպես աղբ: Լիովին ատոմիկ դարձնելու համար կարելի է օգտագործել `MULTI/EXEC` կամ Lua script, կամ `SET key 0 EX 60 NX`, հետո `INCR`:

ՈՐՏԵՂ `wallet/services/redis.ts:56`

ՄԻՋԻՆ **Fixed window-ի թույլ կողմը ո՞րն է:**

Պատուհանի սահմանին burst. 10 request 0:59-ին և 10 request 1:00-ին, այսինքն 20 request մեկ վայրկյանում: Sliding window-ը (Redis sorted set-ով) կամ token bucket-ը սա լուծում են: Առաջադրանքը հենց `INCR + EXPIRE` էր պահանջում:

Cache-aside

wallet/modules/transactions/service.ts — տողեր 15–35

```ts
  public async getHistory(userId: number, page: number, limit: number) {
    if (page === 1) {
      const cached = await redis.get<unknown>(firstPageCacheKey(userId, limit));
      if (cached) return cached;
    }

    const result = await this.repository.getAll({
      userId, page, limit
    });
    const payload = {
      total: result.count,
      page,
      limit,
      transactions: result.rows,
    };

    if (page === 1) {
      await redis.setWithTTL(firstPageCacheKey(userId, limit), payload,
TRANSACTIONS_PAGE_CACHE_TTL_SECONDS);
    }

    return payload;
```

ՀԻՄՆԱԿԱՆ **Ի՞նչ է cache-aside-ը, և ինչո՞ւ միայն առաջին էջը:**

Cache-aside (lazy loading). նախ նայում ենք cache-ում, և եթե կա, վերադարձնում ենք: Եթե չկա, կարդում ենք DB-ից, գրում cache-ում TTL-ով և վերադարձնում: DB-ն cache-ի մասին ոչինչ չգիտի, cache-ը կառավարում է application-ը:

Առաջին էջը ամենահաճախ հարցվողն է (ամեն անգամ, երբ user-ը բացում է app-ը), իսկ հին էջերը հազվադեպ են պետք: Key-ի մեջ կա `limit`-ը, ուստի limit=20 և limit=200 տարբեր cache են:

ՈՐՏԵՂ `wallet/modules/transactions/service.ts:9` `wallet/utils/variables.ts:33`

ՄԻՋԻՆ **Deposit-ից հետո history-ն 30 վայրկյան հին տվյալ ցույց կտա՞:**

Այո, cache-ը invalidate չենք անում deposit-ի ժամանակ: Սա գիտակցված trade-off է (առաջադրանքը պահանջում էր TTL 30s): UI-ում դա լուծված է՝ նոր transaction-ը local merge է արվում, և Live event-ներից անմիջապես երևում է: Ավելի ճիշտ լուծումը կլիներ deposit/transfer-ից հետո `DEL transactions:page1:<userId>:*` անելը:

ՈՐՏԵՂ `web/src/components/Dashboard.tsx:84`

06 RabbitMQ

Wallet-service-ը միայն publish է անում, իսկ notification-service-ը միայն consume: Նրանք իրար մասին չգիտեն:

wallet/services/rabbitmq.ts — տողեր 9-34

```
9     public async connect(): Promise<void> {
10      if (this.channel) return;
11      this.connection = await amqplib.connect(RABBITMQ_URL);
12      this.channel = await this.connection.createChannel();
13      await this.channel.assertExchange(DEPOSIT_EXCHANGE, 'topic', {
14        durable: true
15      });
16
17      this.connection.on('error', (err) => logger.error('RabbitMQ connection error', err));
18      this.connection.on('close', () => {
19        logger.warn('RabbitMQ connection closed');
20        this.channel = null;
21        this.connection = null;
22      });
23
24      logger.info('Connected to RabbitMQ');
25    }
26
27    public async publish(routingKey: string, payload: unknown): Promise<void> {
28      if (!this.channel) await this.connect();
29      this.channel!.publish(
30        DEPOSIT_EXCHANGE,
31        routingKey,
32        Buffer.from(JSON.stringify(payload)),
33        {
34          persistent: true, contentType: 'application/json'
```

notification/index.ts · consumer, retry, DLQ — տողեր 27-77

```
27  async function main() {
28    const connection = await amqplib.connect(RABBITMQ_URL);
29    const channel = await connection.createChannel();
30
31    await channel.assertExchange(DEPOSIT_EXCHANGE, 'topic', {
32      durable: true
33    });
34    await channel.assertQueue(NOTIFICATION_QUEUE, {
35      durable: true
36    });
37    await channel.bindQueue(NOTIFICATION_QUEUE, DEPOSIT_EXCHANGE, DEPOSIT_ROUTING_KEY);
38    await channel.assertQueue(DEAD_LETTER_QUEUE, {
39      durable: true
40    });
41
42    await channel.prefetch(1);
43
44    logger.info(`Waiting for '${DEPOSIT_ROUTING_KEY}' events on '${NOTIFICATION_QUEUE}'...`);
45
46    channel.consume(NOTIFICATION_QUEUE, async (msg: ConsumeMessage | null) => {
47      if (!msg) return;
48
49      const priorAttempts = Number(msg.properties.headers?.['x-retry-count'] ?? 0);
50      const attemptNumber = priorAttempts + 1;
51
52      try {
53        const payload = JSON.parse(msg.content.toString());
54        await simulateSendEmail(payload);
```

```
55        channel.ack(msg);
56      } catch (err) {
57        logger.warn(`Attempt ${attemptNumber}/${MAX_RETRY_ATTEMPTS} failed: ${(err as Error).message}`);
58
59        if (attemptNumber < MAX_RETRY_ATTEMPTS) {
60          channel.ack(msg);
61          await sleep(RETRY_DELAY_MS);
62          channel.sendToQueue(NOTIFICATION_QUEUE, msg.content, {
63            persistent: true,
64            headers:    {
65              ...msg.properties.headers, 'x-retry-count': attemptNumber
66            },
67          });
68        } else {
69          logger.error(`Exhausted ${MAX_RETRY_ATTEMPTS} attempts — routing to ${DEAD_LETTER_QUEUE}`);
70          channel.ack(msg);
71          channel.sendToQueue(DEAD_LETTER_QUEUE, msg.content, {
72            persistent: true, headers: msg.properties.headers
73          });
74        }
75      }
76    }, {
77      noAck: false
```

ՏԵՐՄԻՆ	ՄԵՋ ՄՈՏ	ԲԱՑԱՏՐՈՒԹՅՈՒՆ
Exchange	`deposit_exchange`, type **topic**, durable	Ընդունում է message-ը և routing key-ով որոշում, թե որ queue-ին ուղարկի: Topic-ը թույլ է տալիս pattern-ներ (`deposit.*`)
Routing key	`deposit.created`	Message-ի «հասցեն»
Queue	`notification_queue`, durable	Bind է արված exchange-ին `deposit.created` key-ով
Persistent message	`persistent: true`	Message-ը գրվում է դիսկ, RabbitMQ-ի restart-ից հետո չի կորչում (durable queue-ի հետ միասին)
Manual ack	`noAck: false` + `channel.ack`	Message-ը queue-ից ջնջվում է միայն ack-ից հետո: Consumer-ի crash-ի դեպքում RabbitMQ-ն կուղարկի նորից
Prefetch	`prefetch(1)`	Consumer-ը միաժամանակ ստանում է միայն 1 un-acked message (fair dispatch)
DLQ	`deposit_dlq`	3 անհաջող փորձից հետո message-ը գնում է այստեղ ձեռքով ստուգելու համար

ՀԻՄՆԱԿԱՆ **Ինչո՞ւ RabbitMQ, ոչ թե ուղիղ HTTP call notification-service-ին:**

Decoupling. deposit-ը կախված չէ նրանից, արդյոք notification-service-ը աշխատում է: Եթե այն անջատված է, message-ները սպասում են queue-ում և կմշակվեն, երբ այն միանա: Deposit-ի պատասխանի արագությունն էլ կախված չէ email-ի 1 վայրկյանից: Ապագայում կարելի է ավելացնել նոր consumer-ներ (analytics, fraud) առանց wallet-service-ը փոխելու. պարզապես նոր queue bind անել նույն exchange-ին:

ՈՐՏԵՂ `wallet/modules/deposits/service.ts:67`

ՀԻՄՆԱԿԱՆ **Ինչպե՞ս է աշխատում retry-ը և DLQ-ն:**

Email-ը fail է լինում 20% հավանականությամբ (simulate): Fail-ի դեպքում կարդում ենք `x-retry-count` header-ը: Եթե փորձերը < 3, `ack` ենք անում հին message-ը, սպասում 2 վայրկյան և ուղարկում **նոր** message queue-ի վերջ՝ `x-retry-count + 1`-ով: 3-րդ fail-ից հետո ուղարկում ենք `deposit_dlq`: DLQ-ի message-ները կարելի է տեսնել RabbitMQ management UI-ում (`localhost:15672`):

ՈՐՏԵՂ `notification/index.ts:14` `notification/index.ts:49` `notification/index.ts:71`

ԽՈՐԸ **Ինչո՞ւ ack, հետո sendToQueue, և ոչ nack(requeue):**

`nack(msg, false, true)`-ը message-ը վերադարձնում է queue, բայց **նույն header-ներով**, ուստի չենք կարող հաշվել փորձերը, և այն կարող է անվերջ պտտվել: Մենք նոր message ենք ուղարկում թարմացված counter-ով:

> ՈՒՇԱԴՐՈՒԹՅՈՒՆ
>
> Ազնիվ թույլ կետ. ack-ը և sendToQueue-ն ատոմիկ չեն: Եթե process-ը crash լինի նրանց միջև (մանավանդ 2 վայրկյան sleep-ի ընթացքում), message-ը կկորչի: Ավելի ճիշտ է RabbitMQ-ի native **DLX** (`x-dead-letter-exchange`) + TTL-ով retry queue, կամ նախ publish անել, հետո ack:

ՈՐՏԵՂ `notification/index.ts:60` `notification/index.ts:61`

ԽՈՐԸ **Ի՞նչ կլինի, եթե RabbitMQ-ն անջատված է deposit-ի պահին:**

Deposit-ն արդեն COMMIT է եղել. publish-ը արվում է transaction-ից **հետո** և fire-and-forget է (`.catch`-ով log): Deposit-ը հաջող կլինի, բայց event-ը կկորչի, և email չի գնա: Սա գիտակցված որոշում է. notification-ի խնդիրը չպետք է rollback անի փողը: Event-ը երաշխավորելու ճիշտ լուծումը **Transactional Outbox**-ն է. event-ը գրում ենք DB-ի `outbox` table-ում նույն transaction-ում, իսկ առանձին process-ը publish է անում:

ՈՐՏԵՂ `wallet/modules/deposits/service.ts:76`

ՄԻՋԻՆ **Ինչո՞ւ է notification-service-ը process.exit անում connection-ի կորստի դեպքում:**

Որպեսզի Docker-ը (կամ Kubernetes-ը) restart անի այն մաքուր վիճակով, և այն նորից միանա ու շարունակի consume անել: Reconnect logic ձեռքով գրելը ավելի բարդ է և հեշտ է սխալվել:

ՈՐՏԵՂ `notification/index.ts:85`

07 Streams և backpressure

CSV export-ը տասնյակ հազարավոր row-ի համար, առանց ամբողջը memory-ում պահելու:

wallet/modules/transactions/router.ts — տողեր 24–74

```
24        path:    '/:userId/export',
25        handler: async (req, res) => {
26          const userId = Number(req.params.userId);
27          if (!userId) throw new BadRequest('userId must be a number');
28
29          const client = await pgPool.connect();
30          const queryStream = new QueryStream(
31            `SELECT id, user_id, amount, type, balance_after, created_at
32               FROM transactions
33              WHERE user_id = $1
34              ORDER BY created_at DESC`,
35            [userId],
36            {
37              batchSize: 500
38            }
39          );
40
41          res.setHeader('Content-Type', 'text/csv');
42          res.setHeader('Content-Disposition', `attachment; filename="transactions-${userId}.csv"`);
43
44          const dbStream = client.query(queryStream);
45          const csv = new CsvTransform();
46
47          await new Promise<void>((resolve) => {
48            let released = false;
49            const cleanup = () => {
50              if (released) return;
51              released = true;
52              client.release();
53            };
54
55            dbStream.on('error', (err: Error) => {
56              logger.error('CSV export query stream failed', err);
57              cleanup();
58              if (!res.headersSent) res.status(500);
59              res.end();
60              resolve();
61            });
62            csv.on('error', cleanup);
63
64            res.once('finish', () => {
65              cleanup();
66              resolve();
67            });
68            res.once('close', () => {
69              dbStream.destroy();
70              cleanup();
71              resolve();
72            });
73
74            dbStream.pipe(csv).pipe(res);
```

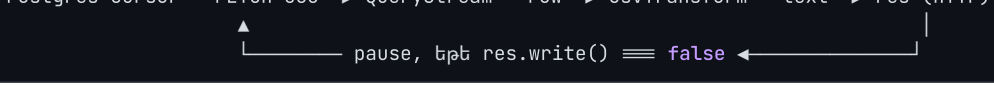
wallet/utils/csvTransform.ts — տողեր 19–35

```ts
export class CsvTransform extends Transform {
  private headerWritten = false;

  constructor() {
    super({
      objectMode: true
    });
  }

  public _transform(row: Record<string, unknown>, _enc: BufferEncoding, callback: TransformCallback): void {
    let chunk = '';
    if (!this.headerWritten) {
      chunk += `${Object.keys(row).join(',')}\n`;
      this.headerWritten = true;
    }
    chunk += `${Object.values(row).map(escapeCsvValue).join(',')}\n`;
    callback(null, chunk);
```

Backpressure chain



ՀԻՄՆԱԿԱՆ **Ինչո՞ւ CSV export-ի համար Sequelize չենք օգտագործում:**

Sequelize-ի `findAll`-ը ամբողջ արդյունքը բեռնում է JS array-ի մեջ: 50 000 row-ը մեծ memory է և երկար սպասում մինչև առաջին byte-ը: `pg-query-stream`-ը Postgres-ում բացում է **server-side cursor** և row-երը բերում է 500-ական (`batchSize`):

ՈՐՏԵՂ `wallet/services/pgPool.ts:5` `wallet/modules/transactions/router.ts:37`

ՀԻՄՆԱԿԱՆ **Ի՞նչ է backpressure-ը, և որտե՞ղ է այն ձեր կոդում:**

Երբ consumer-ը (դանդաղ client) չի հասցնում կարդալ այնքան արագ, որքան producer-ը (DB) գրում է, տվյալը կուտակվում է memory-ում: Backpressure-ը մեխանիզմ է, որ producer-ը «սպասի»:

Մեզ մոտ դա ավտոմատ է `.pipe()`-ի շնորհիվ. երբ `res.write()`-ը վերադարձնում է `false` (socket buffer-ը լիքն է), pipe-ը pause է անում CsvTransform-ը, որը pause է անում QueryStream-ը, և հաջորդ `FETCH`-ը չի ուղարկվում: Երբ socket-ը `drain` է անում, ամեն ինչ resume է լինում: Մենք ձեռքով backpressure կոդ չենք գրել, դա հենց pipe-ի իմաստն է:

ՈՐՏԵՂ `wallet/modules/transactions/router.ts:74` `wallet/utils/csvTransform.ts:15`

ՄԻՋԻՆ **Ի՞նչ կլինի, եթե client-ը download-ի կեսին փակի browser-ը:**

`res`-ը emit է անում `close`: Մենք կանչում ենք `dbStream.destroy()` (cursor-ը փակվում է, Postgres-ը դադարում է աշխատել) և `client.release()`, որ connection-ը վերադառնա pool: `released` flag-ը թույլ չի տալիս release անել երկու անգամ: Առանց սրա pool-ի connection-ները կարող էին «արտահոսել», և pool-ը կսպառվեր:

ՈՐՏԵՂ `wallet/modules/transactions/router.ts:68` `wallet/modules/transactions/router.ts:50`

ՄԻՋԻՆ **Ինչո՞ւ է CsvTransform-ը objectMode-ով:**

Input-ը JS object-ներ են (DB-ի row-եր), ոչ Buffer/string: `objectMode: true`-ի դեպքում ամեն chunk մեկ object է, և highWaterMark-ը հաշվվում է object-ների քանակով: Output-ը string է: Header-ը գրում ենք առաջին row-ի key-երից, իսկ ստորակետ/չակերտ պարունակող արժեքները escape ենք անում CSV-ի կանոններով:

ՈՐՏԵՂ `wallet/utils/csvTransform.ts:24` `wallet/utils/csvTransform.ts:3`

08 Worker Threads և Cluster

Երկուսն էլ «զուգահեռություն» են, բայց տարբեր խնդիրների համար:

	WORKER THREAD	CLUSTER
Ինչ է	Նոր thread նույն process-ի ներսում, իր V8 isolate-ով և event loop-ով	Նոր OS process (`fork`), ամբողջական Node instance
Memory	Կարող է կիսել `SharedArrayBuffer` , հաղորդակցվում է `postMessage` -ով	Ամբողջովին առանձին, հաղորդակցություն IPC-ով կամ Redis-ով
Ինչի համար	CPU-ծանր մեկ task (PDF report)	Ավելի շատ HTTP request սպասարկել, օգտագործել բոլոր CPU-ները
Մեզ մոտ	`wallet/modules/reports/service.ts:26`	`wallet/index.ts:42`

wallet/modules/reports/service.ts — տողեր 19–57

```ts
    public async generate(userId: number): Promise<string> {
     const reportId = uuid();
     await repository.setStatus(reportId, {
       status: 'processing'
     });

     const workerPath = path.join(__dirname, `worker.${EXTENSION}`);
     const worker = new Worker(workerPath, {
       workerData: {
         reportId,
         userId,
         reportsDir: REPORTS_DIR,
         pg:         {
           host: PG_HOST, port: PG_PORT, user: PG_USER, password: PG_PASS, database: PG_DATABASE
         },
       },
       execArgv: EXTENSION === 'ts' ? ['-r', 'ts-node/register'] : undefined,
     });

     worker.on('message', async (msg: { status: 'done' | 'error'; path?: string; message?: string }) => {
       if (msg.status === 'done') {
         await repository.setStatus(reportId, {
           status: 'done', path: msg.path
         });
       } else {
         await repository.setStatus(reportId, {
           status: 'error', message: msg.message
         });
       }
     });

     worker.on('error', async (err) => {
       logger.error(`Report worker crashed for reportId=${reportId}`, err);
       await repository.setStatus(reportId, {
         status: 'error', message: err.message
       });
     });

     return reportId;
```

wallet/modules/reports/worker.ts — տողեր 27–32

```
27    // Simulated CPU-intensive analytics step — this is exactly why this
28    // work happens off the main thread: a synchronous busy-loop here would
29    // otherwise freeze every other request the Wallet Service is serving.
30    let acc = 0;
31    for (let i = 0; i < 2_000_000_00; i += 1) {
32      acc += Math.sqrt(i % 97);
```

ՀԻՄՆԱԿԱՆ **Ինչո՞ւ Worker Thread, և ոչ պարզապես async/await:**

`async/await`-ը օգնում է միայն **I/O**-ի սպասման ժամանակ (DB, network). event loop-ը ազատ է, մինչ սպասում ենք: CPU-ծանր sync կոդը (200 միլիոն iteration-անոց loop) **բլոկում է** event loop-ը. այդ ընթացքում process-ը ոչ մի այլ request չի սպասարկում, նույնիսկ `/health`-ը: Worker Thread-ը այդ loop-ն աշխատեցնում է առանձին thread-ում, իսկ main thread-ը շարունակում է պատասխանել:

Demo. report generate անելու ընթացքում կանչիր `curl /health`, և այն անմիջապես կպատասխանի:

ՈՐՏԵՂ `wallet/modules/reports/worker.ts:31` `ARCHITECTURE.md:29`

ՀԻՄՆԱԿԱՆ **Ինչո՞ւ է endpoint-ը վերադարձնում 202, ոչ 200:**

`202 Accepted` նշանակում է «request-ը ընդունված է, բայց մշակումը դեռ չի ավարտվել»: Client-ը ստանում է `reportId` և poll է անում `GET /reports/:id/status`-ը, մինչև ստանա `done`:

ՈՐՏԵՂ `wallet/modules/reports/router.ts:17`

ՄԻՋԻՆ **Ինչո՞ւ է report-ի status-ը Redis-ում, ոչ JS Map-ում:**

Cluster-ի պատճառով: `POST /generate`-ը կարող է գնալ worker A-ին, իսկ `GET /status`-ը՝ worker B-ին: Դրանք տարբեր process-եր են՝ տարբեր memory-ով: A-ի Map-ը B-ն չի տեսնի: Redis-ը բոլորի համար ընդհանուր է:

ՈՐՏԵՂ `wallet/modules/reports/repository.ts:18` `wallet/modules/reports/service.ts:16`

ՄԻՋԻՆ **Ինչո՞ւ է worker.ts-ը raw pg.Client բացում, և ոչ Sequelize model-ը:**

Worker Thread-ը առանձին JS միջավայր է. main thread-ի Sequelize instance-ը և connection pool-ը այնտեղ հասանելի չեն (object-ները thread-երի միջև չեն կիսվում): Worker-ը ստանում է DB-ի config-ը `workerData`-ով և բացում իր կարճաժամկետ `pg.Client`-ը, որը query-ից հետո փակում է:

ՈՐՏԵՂ `wallet/modules/reports/worker.ts:17` `wallet/modules/reports/service.ts:27`

wallet/index.ts — տողեր 31–61

```
31  async function runMigrationsThenFork(numWorkers: number): Promise<void> {
32    const { database } = await import('./database');
33    // Every cluster worker is a separate OS process that re-executes this
34    // file from scratch, so `sequelize.sync({ alter: true })` running in each
35    // of them concurrently races over the same ALTER/ADD CONSTRAINT
36    // statements. Doing it once here, before any worker exists, avoids that.
37    await database.init({
38      sync: true
39    });
40    await database.connection.close();
41
```

```
42    for (let i = 0; i < numWorkers; i += 1) cluster.fork();
43
44    // Self-healing: a crashed worker never leaves the pool short-staffed.
45    cluster.on('exit', (worker, code, signal) => {
46      logger.warn(
47        `Worker ${worker.process.pid} exited (code=${code}, signal=${signal}). Forking a replacement.`
48      );
49      cluster.fork();
50    });
51  }
52
53  const numWorkers = CLUSTER_WORKERS ?? os.cpus().length;
54
55  if (cluster.isPrimary) {
56    logger.info(`Primary ${process.pid} starting ${numWorkers} worker process(es)`);
57
58    runMigrationsThenFork(numWorkers).catch((err) => {
59      logger.error('Primary failed to run migrations', err);
60      // eslint-disable-next-line no-process-exit
61      process.exit(1);
```

ՀԻՄՆԱԿԱՆ **Ինչպե՞ս է աշխատում Cluster-ը, և ինչպե՞ս են worker-ները կիսում նույն port-ը:**

Primary process-ը չի սպասարկում HTTP. այն `fork()` է անում N worker (`CLUSTER_WORKERS` կամ `os.cpus().length`): Ամեն worker-ը կանչում է `listen(3000)`: Node-ի cluster module-ը port-ը բացում է primary-ում և incoming connection-ները բաժանում worker-ներին (Linux/macOS-ում default-ը round-robin է): Docker-ում `CLUSTER_WORKERS: 2`:

ՈՐՏԵՂ `wallet/index.ts:53` `docker-compose.yml:46`

ՀԻՄՆԱԿԱՆ **Ինչպե՞ս է աշխատում self-healing-ը:**

`cluster.on('exit')`-ը կանչվում է, երբ worker-ը մեռնում է (crash, `kill -9`): Մենք log ենք անում և անմիջապես `cluster.fork()` նոր worker: Այդ ընթացքում մյուս worker-ները շարունակում են սպասարկել request-ները, և `/health`-ը չի ընդհատվում:

ՈՐՏԵՂ `wallet/index.ts:45` `wallet/index.ts:47`

ԽՈՐԸ **Ինչո՞ւ է DB sync-ը կատարվում primary-ում, ոչ ամեն worker-ում:**

Եթե 8 worker միաժամանակ կատարեին `sync({ alter: true })`, նրանք կմրցեին նույն `ALTER TABLE`-ների համար, և կլինեին error-ներ կամ կրկնվող constraint-ներ: Primary-ն մեկ անգամ sync է անում, փակում connection-ը, հետո fork անում: Worker-ները `init({ sync: false })` են անում և միայն միանում են:

ՈՐՏԵՂ `wallet/index.ts:38` `wallet/index.ts:20`

09 Auth, transfer, server

Այն մասերը, որոնք առաջադրանքից դուրս են, բայց UI-ի համար ավելացրել ենք:

wallet/core/auth.middleware.ts — տողեր 7–31

```ts
function extractToken(header: string | undefined): string | undefined {
  if (!header) return undefined;
  let token = header.trim();
  while (/^Bearer\s+/i.test(token)) {
    token = token.replace(/^Bearer\s+/i, '');
  }
  return token || undefined;
}

export const authGuard: MiddlewareHandler = async (req, _res, next) => {
  const token = extractToken(req.headers.authorization);

  if (!token) {
    trace('warn', 'auth.rejected', `${req.method} ${req.originalUrl}: missing Authorization header → 401`);
    next(new Unauthorized('Missing Authorization header'));
    return;
  }

  try {
    const { userId } = verifyToken(token);
    req.authUserId = userId;
    next();
  } catch {
    trace('warn', 'auth.rejected', `${req.method} ${req.originalUrl}: invalid or expired token → 401`);
    next(new Unauthorized('Invalid or expired token'));
```

ՀԻՄՆԱԿԱՆ **Ինչպե՞ս է աշխատում login-ը և JWT-ն:**

Demo login. username-ը պետք է լինի `user<N>` և password-ը նույնը: Ստուգում ենք, որ account-ը գոյություն ունի, և ստորագրում JWT `{ userId }` payload-ով, 1 ժամ ժամկետով: Հետագա request-ներում `Authorization: Bearer <token>`: `authGuard`-ը ստուգում է ստորագրությունը և ժամկետը և դնում `req.authUserId`:

Production-ում կլինեն իրական user-ներ և bcrypt-ով hash արված password-ներ, իսկ secret-ը միայն env-ից կլինի (ոչ default fallback):

ՈՐՏԵՂ `wallet/modules/auth/service.ts:8` `wallet/utils/jwt.ts:4` `wallet/core/auth.middleware.ts:27`

ՀԻՄՆԱԿԱՆ **Ինչո՞ւ է transfer-ում sender-ը token-ից, ոչ body-ից:**

Եթե `fromUserId`-ը body-ից վերցնեինք, ցանկացած մեկը կարող էր ուղարկել ուրիշի փողը: Token-ը ստորագրված է server-ի secret-ով, ուստի `authUserId`-ին կարելի է վստահել: Body-ից վերցնում ենք միայն `toUserId`-ը և `amount`-ը:

ՈՐՏԵՂ `wallet/modules/transfers/router.ts:19`

ՄԻՋԻՆ **Ինչո՞ւ է extractToken-ը while loop-ով «Bearer»-ը կտրում:**

Swagger UI-ի «Authorize» dialog-ը ինքն է ավելացնում `Bearer `: Եթե user-ը paste անի արդեն «Bearer xxx», header-ը կլինի `Bearer Bearer xxx`: Loop-ը կտրում է բոլոր prefix-ները, ուստի «invalid signature» error չի լինում:

ՈՐՏԵՂ `wallet/core/auth.middleware.ts:10`

Server-ի կառուցվածքը

wallet/server.ts — տողեր 62-91

```ts
  public async loadModules(): Promise<void> {
    const modulesUrl = `${__dirname}/modules/`;
    const folders = fs.readdirSync(modulesUrl).filter((folder) => !folder.startsWith('.'));
    folders.forEach((module) => {
      const routerPath = path.join(__dirname, 'modules', module, `router.${EXTENSION}`);
      if (fs.existsSync(routerPath)) {
        this.modules.push({
          name: module, path: routerPath
        });
      } else {
        logger.error('Router path not found: ' + routerPath);
      }
    });
    this.signRoutes();
  }

  private signRoutes() {
    for (const moduleObj of this.modules) {
      // eslint-disable-next-line @typescript-eslint/no-var-requires
      const { module } = require(moduleObj.path);
      const moduleRouter = Router();
      if (module.middleware) moduleRouter.use(module.middleware);
      module.routes.forEach((route: SingleRoute) => {
        const { method, path: routePath, actions } = this.createRoute(route);
        moduleRouter[method](routePath, actions);
      });
      const modulePath = module.path && module.path !== ''
        ? module.path
        : `/${convertModuleToRoute(moduleObj.name)}`;
      this.router.use(modulePath, moduleRouter);
```

ՀԻՄՆԱԿԱՆ **Ինչպե՞ս են route-երը գրանցվում. ես չեմ տեսնում app.post("/deposit"):**

Auto-loader. `loadModules()`-ը կարդում է `modules/`-ի բոլոր folder-ները, ամեն մեկից `require` անում `router.ts`-ը և ստանում `{ path, routes }` object-ը: `signRoutes()`-ը ամեն route-ի համար կառուցում է chain՝ middleware-ներ → handler → response. Նոր module ավելացնելու համար բավական է ստեղծել folder `router.ts`-ով:

ՈՐՏԵՂ `wallet/server.ts:62` `wallet/server.ts:78`

ՀԻՄՆԱԿԱՆ **Ինչպե՞ս են error-ները վերածվում HTTP status-ի:**

Ունենք error class-եր (`BadRequest` 400, `Unauthorized` 401, `NotFound` 404, `Conflict` 409…), ամեն մեկն իր `status`-ով: Handler-ը throw է անում, wrapper-ը այն փոխանցում է `next(err)`-ին, իսկ global error middleware-ը վերադարձնում է `{ ok: false, status, message }`: Չճանաչված error-ը վերադարձնում է 500:

ՈՐՏԵՂ `wallet/utils/errors.ts:19` `wallet/server.ts:45`

ԽՈՐԸ **Ի՞նչ է AsyncLocalStorage-ը trace.ts-ում:**

Դա Node-ի մեխանիզմ է՝ «request-ի context» պահելու առանց այն ամեն ֆունկցիայի parameter-ով փոխանցելու: Middleware-ը request-ի սկզբում դնում է `{ traceId, fault }`, իսկ ցանկացած խորը service-ում `trace()`-ը կարող է այն կարդալ: Այս կերպ Logs էջում մեկ request-ի բոլոր event-ները խմբավորվում են նույն գույնով:

ՈՐՏԵՂ `wallet/utils/trace.ts:29` `wallet/core/trace.middleware.ts:10`

10 Ֆայլ առ ֆայլ

Ամեն ֆայլի դերը, թե ինչ է անում, և այն տողերը, որոնց մասին կարող են հարցնել «այս տողը ինչ է անում»:

Wallet-service · մուտք և ենթակառուցվածք

`wallet/index.ts`

Entry point

Process-ը սկսվում է այստեղից: Եթե սա primary process-ն է, մեկ անգամ սինքրոնացնում է DB schema-ն, փակում connection-ը և fork է անում N worker: Եթե worker է, միանում է DB-ին, Redis-ին, RabbitMQ-ին, բեռնում module-ները և լսում port 3000-ը:

`wallet/index.ts:55`	Որոշում է՝ primary է, թե worker: Նույն ֆայլը կատարվում է երկու դերում էլ:
`wallet/index.ts:38`	Միայն primary-ն է schema-ն փոխում, որ worker-ները չմրցեն ALTER TABLE-ի համար:
`wallet/index.ts:45`	Self-healing. մեռած worker-ի փոխարեն անմիջապես նորն է fork արվում:
`wallet/index.ts:20`	Worker-ը միայն միանում է, schema-ին չի դիպչում:

`wallet/server.ts`

Express app

Ստեղծում է Express app-ը, միացնում է ընդհանուր middleware-ները (cors, json, morgan, trace), `/health`-ը և `/docs`-ը: Ավտոմատ գտնում է բոլոր `modules/*/router.ts`-երը և գրանցում նրանց route-երը: Վերջում global error handler-ն է:

`wallet/server.ts:33`	Ամեն request-ի համար բացում է trace context (Logs/Test lab):
`wallet/server.ts:39`	Swagger UI. API-ն browser-ից փորձելու համար:
`wallet/server.ts:78`	Ամեն module-ի route-ի համար կառուցում է chain՝ middleware → handler → JSON response:
`wallet/server.ts:45`	Ցանկացած throw արված error վերածվում է `{ ok:false, status, message }`-ի:

`wallet/database.ts`

DB connection և schema

Sequelize connection, ավտոմատ բեռնում է բոլոր `model.ts`-երը և associations-ը: Sync-ից հետո raw SQL-ով ավելացնում է այն, ինչ Sequelize-ը ինքը չի անում:

`wallet/database.ts:33`	Schema-ն թարմացվում է model-ներից (migration չկա):
`wallet/database.ts:69`	CHECK (balance >= 0). DB-ն ինքը չի թողնի բացասական balance:
`wallet/database.ts:70`	Composite index-ը, IF NOT EXISTS-ով, որ երկրորդ անգամ error չտա:

`wallet/services/redis.ts`

Redis client

Redis-ի բարակ wrapper: JSON serialize/parse և մեր պետք եկած ատոմիկ գործողությունները:

`wallet/services/redis.ts:43`	`SET key value NX EX ttl`. idempotency-ի ատոմիկ claim, true եթե հաղթեցինք:
`wallet/services/redis.ts:55`	INCR, և առաջին անգամ EXPIRE. rate limit-ի counter:

wallet/services/rabbitmq.ts

RabbitMQ publisher

Միայն publish է անում: Connect-ի ժամանակ declare է անում topic exchange-ը (durable): Connection-ի կորստի դեպքում հաջորդ publish-ը նորից կմիանա:

wallet/services/rabbitmq.ts:13	Exchange-ը ստեղծվում է, եթե չկա: Notification-service-ն էլ նույնն է declare անում:
wallet/services/rabbitmq.ts:34	Message-ը գրվում է դիսկ, broker-ի restart-ից հետո չի կորչում:

wallet/services/pgPool.ts

Raw pg Pool

Առանձին connection pool միայն CSV export-ի համար, քանի որ Sequelize-ը stream/cursor չի տալիս:

wallet/services/pgPool.ts:9	Pool-ը, որից export-ը վերցնում է client և release անում:

wallet/utils/variables.ts

Կարգավորումներ

Բոլոր env փոփոխականները default արժեքներով մեկ տեղում: Եթե հարցնեն «որտեղ է 10-ը կամ 30 վայրկյանը», այստեղ է:

wallet/utils/variables.ts:27	10 deposit/րոպե/user:
wallet/utils/variables.ts:30	Claim-ի TTL 60s, արդյունքինը՝ 24h:
wallet/utils/variables.ts:33	Page-1 cache 30s:
wallet/utils/variables.ts:40	Test lab-ի X-Fault-ը աշխատում է միայն true-ի դեպքում:

Wallet-service · core (ընդհանուր շերտ)

wallet/core/repository.ts

Generic repository

Base class բոլոր Sequelize repository-ների համար՝ create/update/delete/getOne/getAll: `getAll`-ը paginated է և միշտ sort է անում `createdAt DESC`-ով:

wallet/core/repository.ts:34	Offset pagination. (page - 1) × limit:
wallet/core/repository.ts:36	Մեկ կանչով 2 query՝ էջի row-երը և ընդհանուր COUNT-ը:
wallet/core/repository.ts:40	Համընկնում է composite index-ի հետ, ուստի sort չի պահանջվում:

wallet/core/idempotency.middleware.ts

Idempotency middleware

Ստուգում է `Idempotency-Key`-ը: Եթե key-ը նոր է, claim է անում, եթե արդեն ավարտված է՝ replay, եթե դեռ ընթացքում է՝ 409: Ունի նաև 2 helper՝ արդյունքը պահելու և claim-ը ազատելու համար:

wallet/core/idempotency.middleware.ts:25	Կա՞ արդեն գրառում այս key-ով:
wallet/core/idempotency.middleware.ts:35	Replay. վերադարձնում ենք պահված պատասխանը, DB-ին չենք դիպչում:
wallet/core/idempotency.middleware.ts:39	Ատոմիկ claim. զուգահեռ request-ներից միայն մեկը կհաղթի:
wallet/core/idempotency.middleware.ts:66	Fail-ի դեպքում key-ը ջնջվում է, որ retry-ը աշխատի:

wallet/core/rateLimiter.middleware.ts

Rate limiter

Fixed window. ամեն user-ի համար ամեն րոպեն առանձին counter է:

wallet/core/rateLimiter.middleware.ts:12	Ընթացիկ րոպեի համարը. նոր րոպե = նոր key:
wallet/core/rateLimiter.middleware.ts:16	11-րդ request-ը ստանում է 429, և DB-ին ոչինչ չի հասնում:

`wallet/core/auth.middleware.ts` — JWT guard

Կարդում է `Authorization` header-ը, ստուգում JWT-ն և դնում `req.authUserId`:

`wallet/core/auth.middleware.ts:10`	Կտրում է կրկնված «Bearer»-ը (Swagger-ի հետ կապված խնդիր):
`wallet/core/auth.middleware.ts:26`	Ստորագրության և ժամկետի ստուգում: Սխալի դեպքում 401:

`wallet/core/associations.ts` — Relation-ներ

Account-ը ունի շատ transaction: Սա DB-ում ստեղծում է FK `transactions.user_id → accounts.user_id` (CASCADE):

`wallet/core/associations.ts:4`	1 → N կապ:
`wallet/core/associations.ts:10`	FK, որ չի թողնի transaction գոյություն չունեցող account-ի համար:

`wallet/core/trace.middleware.ts` — Trace context

Header-ներից վերցնում է `X-Trace-Id`-ն ու `X-Fault`-ը և պահում AsyncLocalStorage-ում: Non-GET request-ի ավարտին գրում է http event:

`wallet/core/trace.middleware.ts:10`	Մեկ request-ի բոլոր event-ները կապվում են այս id-ով:
`wallet/core/trace.middleware.ts:11`	Որ կետում «պայթել» (միայն Test lab-ի համար):

Wallet-service · modules

`wallet/modules/accounts/model.ts` — accounts table

Ամեն user-ի մեկ account՝ `id`, `user_id` (unique), `balance`:

`wallet/modules/accounts/model.ts:27`	Մեկ user-ին միայն մեկ account: Ստեղծում է unique index, որը FOR UPDATE-ի WHERE-ը օգտագործում է:
`wallet/modules/accounts/model.ts:32`	Փողը ճշգրիտ տասնորդական է, ոչ float:

`wallet/modules/transactions/model.ts` — transactions table

History. ամեն deposit/transfer-ի մեկ row՝ գումարը, տեսակը և դրանից հետո balance-ը:

`wallet/modules/transactions/model.ts:40`	Տեսակը սահմանափակ է՝ deposit/withdrawal/transfer_in/transfer_out:
`wallet/modules/transactions/model.ts:52`	History-ն երբեք չի փոփոխվում, միայն ավելանում է:
`wallet/modules/transactions/model.ts:58`	Composite index (user_id, created_at):

`wallet/modules/deposits/router.ts` — POST /deposit

Route-ի սահմանումը. middleware-ների հերթականությունը, validation, service կանչ, արդյունքի պահում idempotency-ի համար:

`wallet/modules/deposits/router.ts:12`	Նախ rate limit, հետո idempotency: Ավելցուկային request-ը key չի «զբաղեցնում»:
`wallet/modules/deposits/router.ts:28`	Հաջող պատասխանը պահվում է 24h. retry-ը կստանա նույնը:
`wallet/modules/deposits/router.ts:31`	Error-ի դեպքում key-ը ազատվում է:

wallet/modules/deposits/service.ts

Deposit-ի logic

Ամբողջ deposit-ը մեկ transaction է՝ lock, UPDATE balance, INSERT history, COMMIT: Հետո publish event:

- `wallet/modules/deposits/service.ts:21` Explicit isolation level:
- `wallet/modules/deposits/service.ts:27` SELECT ... FOR UPDATE. lost update-ի կանխում:
- `wallet/modules/deposits/service.ts:43` Երկրորդ գրառումը նույն t-ով. ատոմիկ:
- `wallet/modules/deposits/service.ts:67` COMMIT-ից հետո, fire-and-forget:

wallet/modules/transfers/service.ts

Transfer-ի logic

Երկու account, երկու lock (ըստ id-ի կարգի), balance-ի ստուգում, 2 UPDATE + 2 INSERT, ամեն ինչ մեկ transaction-ում:

- `wallet/modules/transfers/service.ts:19` Lock ordering. deadlock-ի կանխում:
- `wallet/modules/transfers/service.ts:42` Բավարար չէ → 400 → ROLLBACK:
- `wallet/modules/transfers/service.ts:71` Երկու կողմի համար էլ history row:

wallet/modules/transactions/service.ts

History service

Cache-aside. page 1-ը Redis-ից, եթե կա, այլապես DB-ից, և ապա cache-ում պահում 30s:

- `wallet/modules/transactions/service.ts:17` Cache hit. DB չենք կանչում:
- `wallet/modules/transactions/service.ts:32` Cache miss → DB → cache:

wallet/modules/transactions/router.ts

History և CSV route-եր

GET history (service-ով) և GET export (raw pg cursor → CSV Transform → response):

- `wallet/modules/transactions/router.ts:30` Server-side cursor 500-անոց batch-երով:
- `wallet/modules/transactions/router.ts:74` Pipe chain. backpressure-ն ավտոմատ է:
- `wallet/modules/transactions/router.ts:68` Client-ը կտրեց կապը → cursor-ը փակվում է, connection-ը վերադառնում pool:

wallet/modules/reports/service.ts

Report service

Ստեղծում է reportId, Redis-ում գրում processing, բացում Worker Thread և անմիջապես վերադառնում: Worker-ի message/error-ի դեպքում թարմացնում է status-ը:

- `wallet/modules/reports/service.ts:26` Նոր thread, որ main event loop-ը չբլոկվի:
- `wallet/modules/reports/service.ts:38` postMessage → status done/error Redis-ում:

wallet/modules/reports/worker.ts

Worker-ի կոդը

Աշխատում է առանձին thread-ում. SQL query, CPU-ծանր loop, PDF ֆայլի գրում, հետո postMessage:

- `wallet/modules/reports/worker.ts:17` Սեփական DB connection (main-ի pool-ը հասանելի չէ):
- `wallet/modules/reports/worker.ts:31` Simulated ծանր հաշվարկ. ահա թե ինչու է պետք thread:
- `wallet/modules/reports/worker.ts:62` Արդյունքը main thread-ին:

`wallet/modules/auth/service.ts` — Login

Demo auth. `userN` / `userN`, account-ի ստուգում, JWT 1 ժամով:

- `wallet/modules/auth/service.ts:8` — Username-ից հանում է userId-ն:
- `wallet/modules/auth/service.ts:20` — Token-ի ստորագրում:

`wallet/modules/events/router.ts` — Events API

UI-ի Logs/Test lab-ի համար. վերադարձնում է Redis-ի event-ները `after` id-ից հետո, կամ ջնջում բոլորը:

- `wallet/modules/events/router.ts:16` — Polling-ի համար միայն նոր event-ները:

Մյուսները

`wallet/utils/withDeadlockRetry.ts` — Deadlock retry

Ընդհանուր wrapper. եթե fn-ը նետում է Postgres 40P01, փորձում է նորից (մինչև 3 անգամ, backoff-ով):

- `wallet/utils/withDeadlockRetry.ts:21` — Միայն deadlock-ի դեպքում. մյուս error-ները անմիջապես վեր են գնում:
- `wallet/utils/withDeadlockRetry.ts:24` — 50ms, 100ms...

`wallet/scripts/deadlock-demo.ts` — Deadlock demo

Գիտակցաբար ստեղծում է deadlock՝ 2 transaction, 2 lock, հակառակ հերթականությամբ, 300ms ընդմիջումով:

- `wallet/scripts/deadlock-demo.ts:48` — T1. 1, հետո 2:
- `wallet/scripts/deadlock-demo.ts:49` — T2. 2, հետո 1 → deadlock → retry:

`wallet/utils/csvTransform.ts` — CSV Transform

Object-ներից CSV տող է սարքում: Առաջին row-ից header, escape ստորակետերի և չակերտների համար:

- `wallet/utils/csvTransform.ts:24` — Input-ը JS object է:
- `wallet/utils/csvTransform.ts:3` — CSV-ի ճիշտ escape:

`wallet/utils/trace.ts` — Event log

trace()-ը գրում է event Redis list-ում (cluster-ի բոլոր worker-ների համար ընդհանուր), faultPoint()-ը throw է անում Test lab-ի համար:

- `wallet/utils/trace.ts:29` — Request-ի context առանց parameter-ների:
- `wallet/utils/trace.ts:57` — Պահում է միայն վերջին 1000-ը:
- `wallet/utils/trace.ts:88` — Rollback-ը ստուգելու համար error ընտրված կետում:

`wallet/seeds/index.ts` — Seed

Ստեղծում է userId 1–5 account-ները 0 balance-ով: `findOrCreate`-ի շնորհիվ կարելի է run անել շատ անգամ:

- `wallet/seeds/index.ts:11` — Idempotent. կրկնակի չի ստեղծում:

`notification/index.ts` — Notification consumer

Առանձին process. consume, fake email (1s, 20% fail), ack, retry 3 անգամ, հետո DLQ:

`notification/index.ts:42`	Մեկ-մեկ, fair dispatch:
`notification/index.ts:49`	Փորձերի քանակը header-ից:
`notification/index.ts:71`	3-րդ fail-ից հետո DLQ:

`docker-compose.yml` — Docker stack

Postgres, Redis, RabbitMQ (healthcheck-ներով), wallet-service և notification-service: Service-ները սկսում են միայն infra-ի healthy լինելուց հետո:

`docker-compose.yml:3`	DB image:
`docker-compose.yml:46`	2 worker process:
`docker-compose.yml:62`	Սպասում է infra-ին:

`web/src/lab/scenarios.ts` — Test lab

Պլանավորված error-ների 14 սցենար, որոնք կանչում են իրական API-ն և ստուգում status-ներ, balance-ներ և server trace:

`web/src/lab/scenarios.ts:40`	Idempotency սցենարներ:
`web/src/lab/scenarios.ts:104`	Rollback fault injection-ով:
`web/src/lab/scenarios.ts:176`	Concurrency/double spend:

`web/src/components/Dashboard.tsx` — Wallet UI

Balance, chart, Activity: Poll է անում /events-ը, և նոր commit-ները անմիջապես ավելանում են list-ում:

`web/src/components/Dashboard.tsx:57`	Commit event → transaction row:
`web/src/components/Dashboard.tsx:140`	Live polling:

11 Խորամանկ հարցեր

Եթե գնահատողը լավ է ճանաչում տեխնոլոգիաները, կփնտրի թույլ կողմեր: Լավագույն ռազմավարությունը՝ ազնվորեն ընդունել, բացատրել, թե ինչու է այդպես, և ասել, թե ինչպես կլավացնեիր: Սա ցույց է տալիս, որ հասկանում ես կոդը, ոչ թե պարզապես copy-paste ես արել:

ԽՈՐԸ **accounts.user_id-ի վրա 32 կրկնվող unique index կա: Ինչո՞ւ:**

Սա Sequelize-ի հայտնի bug-ն է. `sync({ alter: true })`-ը `unique: true` այունակի համար ամեն startup-ի ժամանակ `ALTER TABLE … ADD UNIQUE` է անում, և Postgres-ը ստեղծում է նոր constraint նոր անունով (`_key1`, `_key2` …): Ամեն `docker compose up`-ը մեկ հատ է ավելացնում: Վնասը. INSERT/UPDATE-ը դանդաղում է, քանի որ ամեն index-ը պետք է թարմացվի:

Լուծում. production-ում `sync(alter)`-ի փոխարեն օգտագործել **migration**-ներ (sequelize-cli/umzug), կամ unique-ը սահմանել անունով (`indexes: [{ unique: true, fields: ['user_id'], name: 'accounts_user_id_unique' }]`) և մաքրել կրկնվողները:

ՈՐՏԵՂ `wallet/modules/accounts/model.ts:27` `wallet/database.ts:33`

ԽՈՐԸ **Զուգահեռ transfer-ների ժամանակ մի մասը 500 է վերադարձնում:**

`REPEATABLE READ` + `FOR UPDATE`-ի դեպքում, եթե row-ը փոխվել է snapshot-ից հետո, Postgres-ը նետում է `40001` serialization failure: `withDeadlockRetry`-ը բռնում է միայն `40P01`-ը: Փողը չի կորչում (rollback), բայց օրինական request-ը մերժվում է: **Լուծում.** READ COMMITTED (lock-ն արդեն պաշտպանում է) կամ retry նաև `40001`-ի դեպքում:

ՈՐՏԵՂ `wallet/utils/withDeadlockRetry.ts:21` `wallet/modules/transfers/service.ts:26`

ԽՈՐԸ **Validation error-ի դեպքում idempotency key-ը 60 վայրկյան «կախված» է մնում:**

Router-ում `userId` / `amount` validation-ը `try`-ից **առաջ** է: Եթե այն throw անի, `releaseIdempotencyClaim`-ը չի կանչվում, և key-ը մնում է `processing` 60s: Ուղղված request-ը նույն key-ով կստանա 409: **Լուծում.** validation-ը տեղափոխել idempotency middleware-ից առաջ կամ `try`-ի ներս:

ՈՐՏԵՂ `wallet/modules/deposits/router.ts:17` `wallet/modules/deposits/router.ts:19`

ԽՈՐԸ **Idempotency key-ը կապված չէ user-ի կամ body-ի հետ:**

Եթե երկու տարբեր user պատահաբար օգտագործեն նույն key-ը, երկրորդը կստանա առաջինի պատասխանը: Եթե նույն key-ով ուղարկվի ուրիշ `amount`, կվերադարձվի հին պատասխանը, և error չի լինի: **Լուծում.** key-ը կազմել `userId + key`-ից և պահել body-ի hash-ը, mismatch-ի դեպքում վերադարձնել `422`:

ՈՐՏԵՂ `wallet/core/idempotency.middleware.ts:13`

ՄԻՋԻՆ **POST /deposit-ը auth չունի: Ցանկացածը կարող է deposit անել ցանկացածին:**

Այո, ինչպես առաջադրանքում էր. deposit-ը `userId`-ը body-ից է ստանում, և rate limit-ն էլ է body-ի `userId`-ով: Իրական համակարգում deposit-ը կգար payment provider-ի webhook-ից՝ signature ստուգումով, կամ կպահանջեր JWT, ինչպես transfer-ը:

ՈՐՏԵՂ `wallet/modules/deposits/router.ts:14`

ԽՈՐԸ **Event-ը կարող է կորչել deposit-ից հետո:**

Այո, երեք տեղում. (1) publish-ը commit-ից հետո է և fire-and-forget, ուստի եթե RabbitMQ-ն անջատված է կամ process-ը crash լինի commit-ից հետո, event-ը կկորչի. (2) channel-ը confirm mode-ում չէ, այսինքն չգիտենք՝ broker-ը ստացել է. (3) notification-ում ack-ը արվում է նոր message ուղարկելուց առաջ: **Լուծում.** Transactional Outbox, publisher confirms, RabbitMQ DLX:

ՈՐՏԵՂ `wallet/modules/deposits/service.ts:67` `notification/index.ts:60`

ՄԻՋԻՆ **Balance-ը հաշվվում է JS Number-ով:**

`(Number(balance) + amount).toFixed(2)`: 2 տասնորդական նիշի և չափավոր գումարների դեպքում `toFixed`-ը կլորացնում է ճիշտ, բայց մեծ թվերի դեպքում (> 2^{53} cent) ճշգրտությունը կկորչի: **Լուծում.** հաշվարկը SQL-ում (`balance = balance + $1 RETURNING balance`) կամ `decimal.js` / `big.js`:

ՈՐՏԵՂ `wallet/modules/deposits/service.ts:33`

ՄԻՋԻՆ **Ամեն report-ի համար նոր Worker Thread է: 1000 request-ը 1000 thread կլինի՞:**

Այո, limit չկա: Ամեն thread-ն ունի իր V8 isolate-ը (~10MB+) և CPU-ն կծանրաբեռնվի: **Լուծում.** worker pool (օր. `piscina`) ֆիքսված թվով thread-երով և queue-ով, կամ BullMQ concurrency limit-ով: Եվ report-ի status-ը Redis-ում TTL չունի, ուստի պետք է ավելացնել `EX`:

ՈՐՏԵՂ `wallet/modules/reports/service.ts:26` `wallet/modules/reports/repository.ts:18`

ՄԻՋԻՆ **Ավտոմատ test-եր կա՞ն:**

Unit test framework (jest/vitest) չկա: Վարքագիծը ստուգված է (1) `npm run demo:deadlock`-ով, (2) web-ի Test lab-ի 14 սցենարով, որոնք run են լինում իրական API-ի վրա և ստուգում status code-եր, balance-ներ ու server trace, (3) PROJECT_GUIDE-ի curl սցենարներով: Հաջորդ քայլը կլիներ integration test-եր testcontainers-ով (իրական Postgres/Redis Docker-ում):

ՈՐՏԵՂ `web/src/lab/scenarios.ts:37`

ՀԻՄՆԱԿԱՆ **Կոդը AI-ով ես գրել: Ինչպե՞ս ապացուցես, որ հասկանում ես:**

Լավագույն պատասխանը ցույց տալն է: Բացիր Test lab-ը, run արա «Crash after debiting the sender» սցենարը և բացատրիր trace-ը տող առ տող (BEGIN → lock → debit → fault → ROLLBACK): Հետո ասա, թե ինչ թույլ կողմեր ես ինքդ գտել (այս բաժինը): Մարդը, որ միայն copy-paste է արել, չի կարող ասել, թե ինչու է կոդը սխալ կոնկրետ դեպքում:

12 Demo և բառարան

Եթե ասեն «ցույց տուր», ահա պատրաստի հրամանները: Ամենատպավորիչը web-ի **Test lab**-ն է (`#/lab`) և **Logs**-ը (`#/logs`):

Ամբողջ stack-ը

```
npm run all
```

Idempotency (նույն key 2 անգամ)

```
curl -X POST localhost:3000/deposit -H "Content-Type: application/json" \
  -H "Idempotency-Key: demo-1" -d '{"userId":1,"amount":100}'   # 2 անգամ → նույն transactionId
```

Rate limit (11-րդը 429)

```
for i in $(seq 1 11); do curl -s -o /dev/null -w "%{http_code} " -X POST localhost:3000/deposit \
  -H "Content-Type: application/json" -H "Idempotency-Key: rl-$i" -d '{"userId":3,"amount":1}'; done
```

Composite index

```
docker compose exec postgres psql -U postgres -d wallet -c \
  "SET enable_seqscan=off; EXPLAIN SELECT * FROM transactions WHERE user_id=1 ORDER BY created_at DESC LIMIT
20;"
```

CHECK constraint

```
docker compose exec postgres psql -U postgres -d wallet -c "\d accounts"
```

Deadlock + retry

```
cd wallet-service && npm run demo:deadlock
```

CSV stream

```
curl -OJ localhost:3000/transactions/1/export
```

Worker Thread (API-ն չի կախվում)

```
curl -X POST localhost:3000/reports/generate -H "Content-Type: application/json" -d '{"userId":1}'
curl localhost:3000/health          # անմիջապես պատասխանում է
```

Cluster self-healing

```
docker compose exec wallet-service ps aux | grep node
docker compose exec wallet-service kill -9 <worker-pid>
docker compose logs wallet-service --tail 5   # "Forking a replacement"
```

RabbitMQ DLQ

```
open http://localhost:15672    # guest/guest → Queues → deposit_dlq
```

Բառարան

ՏԵՐՄԻՆ	ՊԱՐԶ ԲԱՑԱՏՐՈՒԹՅՈՒՆ
ACID	Atomicity, Consistency, Isolation, Durability. transaction-ի 4 երաշխիքը

ՏԵՐՄԻՆ	ՊԱՐԶ ԲԱՑԱՏՐՈՒԹՅՈՒՆ
Isolation level	Որքան են զուգահեռ transaction-ները «տեսնում» իրար փոփոխությունները
Snapshot	DB-ի «լուսանկարը» որոշակի պահին. REPEATABLE READ-ը ամբողջ transaction-ում մեկն է օգտագործում
Row-level lock	Lock մեկ row-ի վրա (FOR UPDATE), ոչ ամբողջ table-ի
Lost update	Երկու զուգահեռ գրառումից մեկը ջնջում է մյուսին
Deadlock / 40P01	Երկու transaction սպասում են իրար. Postgres-ը մեկին abort է անում
40001	Serialization failure. REPEATABLE READ/SERIALIZABLE-ում կոնֆլիկտ
Lock ordering	Lock-երը միշտ նույն հերթականությամբ վերցնել, որ deadlock չլինի
Composite index	Index մի քանի սյունակի վրա. աշխատում է leftmost prefix-ով
Leftmost prefix	Composite index-ը օգտագործվում է, եթե query-ն ֆիլտրում է ձախից սկսվող սյունակներով
Idempotency	Նույն request-ը N անգամ = նույն արդյունքը, ինչ 1 անգամ
SET NX EX	Redis. գրել միայն եթե key-ը չկա, TTL-ով, ատոմիկ
Cache-aside	App-ը նախ նայում է cache-ում, miss-ի դեպքում DB → cache
Fixed window	Rate limit, որտեղ ժամանակը բաժանված է հավասար պատուհանների
Exchange / routing key	RabbitMQ. ով ընդունում է message-ը / ինչ հասցեով
ACK / prefetch	Հաստատում, որ message-ը մշակված է / քանի un-acked message տալ միաժամանակ
DLQ	Dead Letter Queue. message-ներ, որոնք չհաջողվեց մշակել
Outbox pattern	Event-ը գրել DB-ում նույն transaction-ում, հետո publish, որ չկորչի
Backpressure	Դանդաղ consumer-ը կանգնեցնում է արագ producer-ին
Server-side cursor	DB-ն պահում է query-ի դիրքը, client-ը FETCH-ով վերցնում է մաս-մաս
Worker Thread	Առանձին JS thread CPU-ծանր աշխատանքի համար
Cluster	Մի քանի Node process մեկ port-ի վրա
Event loop blocking	Sync CPU կոդը կանգնեցնում է բոլոր մյուս request-ները
JWT	Ստորագրված token, որ server-ը ստուգում է առանց DB-ի
Fault injection	Գիտակցաբար error առաջացնել, որ ստուգենք rollback-ը